CS 4/5789 taught by Sarah Dean

Reinforcement Learning

FALL 2026

Cornell University Computer Science Department

Contents

1 *Policies, Optimization, and Gradients* 3

2 *Sequential Decisions* 18

3 *Optimality and Value Iteration* 29

4 *Iterative Policy Improvement* 39

5 *Online Decision-making* 52

6 *Receding Horizons* 62

Incomplete lecture notes developed and used by Sarah Dean to teach CS 4/5789 at Cornell University in fall 2026. Complete draft to come in the spring.

This version posted November 2026

1
Policies, Optimization, and Gradients

We begin our study of reinforcement learning by jumping into the two core questions. First, given observed information about the world, which action should we take? We answer this question by reasoning about decision *policies*, which map observations to actions. Second, how do we design good policies? To answer this, we must define what it means to make good decisions; we take an empirical rather than philosophical approach. It is assumed that we can observe a *reward* signal and that a policy is better if it achieves higher reward. However, we observe *only* the reward of actions taken, rather than labels of the best action or counterfactual outcomes, requiring careful thought about how to *estimate* the reward a policy achieves. Eventually, we turn policy design into an *optimization* problem, which we primarily tackle using first-order methods, i.e. algorithms based on estimates of the *gradient* of our objective function.

1.1 Setting

The decision making entity is referred to as "the agent", and the setting in which it acts is called "the environment". The interaction between an agent and the environment is as follows.

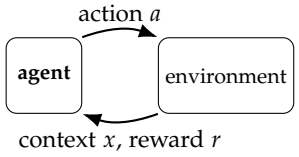


Figure 1.1: The interaction loop.

The agent observes information about the environment, referred to as the context for the decision. The context is denoted as $x \in \mathcal{X}$, where $\mathcal{X}$ is the set of all possible contexts. The agent takes an action $a \in \mathcal{A}$, where $\mathcal{A}$ is the set of all possible actions. After taking an action, the agent observes a reward $r \in \mathbb{R}$ generated by the environment. There is a causal order: first observation, then action, and finally reward. The action can depend on the observation (but not the reward) and the reward can depend on both the context and the observation.

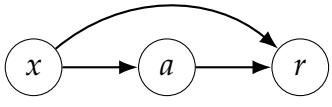


Figure 1.2: Graphical model illustrating dependence of context $x \sim \mathcal{D}_x$, action $a \sim \pi(x)$, and reward $r \sim R(x, a)$.

It is not realistic to expect that the reward is always a deterministic function of context and action. As a result, we take a probabilistic viewpoint and write $r \sim R(x, a)$ where $R : \mathcal{X} \times \mathcal{A} \to \Delta(\mathbb{R})$ is

the reward function. We do not assume that we know the function $R(x, a)$ a priori.

Example 1.1 (Recommendation)**.** Consider movie recommendation on a streaming platform. The context x includes user profile information and viewing history. The action a is the new movie to promote on the front page, and $\mathcal{A}$ is the set of all recent movies available on the platform. The reward r is whether the user clicks $r = 1$ or not $r = 0$ on the recommended movie.

Alternative reward definitions are possible: whether the user eventually watches the movie, whether they watch at least 15 minutes of the movie, or the proportion of the movie that they watch. Each definition corresponds to a slightly different platform goal.

Our main focus in reinforcement learning is on the agent; the specification of the environment via the context and reward is outside our scope for now. As such, the main task is to design a (possibly randomized) policy $\pi : \mathcal{X} \to \Delta(\mathcal{A})$ which maps contexts to distributions over actions. That is, the agent samples an action $a \sim \pi(x)$ from the distribution defined by the context and the policy. We will also write $\pi(a|x)$ to mean the probability of taking action a in context x. In the special case that policy is deterministic, $\pi(a|x)$ is equal to one for exactly one action a (and zero for the rest), and we will use the notation $a = \pi(x)$.

We consider a family of parametric policies, similarly to how we consider parametric functions in supervised machine learning. We denote by π_θ the policy instatiated by the parameters $\theta \in \Theta$. Then the policy class is $\Pi = \{\pi_\theta \mid \theta \in \Theta\}$.

Example 1.2 (Recommendation cont.)**.** Continuing the movie recommendation example, we may choose to map user contexts x to movies a with a deep network. The parameters θ are the weights of the network and the policy class Π is defined by the chosen architecture. The user profile information and viewing history in x is represented as vectors or tokens. The network output consists of $|\mathcal{A}|$ nodes equal to the number of possible movies to promote, and given x, the output of each node is between zero and one, representing the probability $\pi(a|x)$, so $\sum_{a\in\mathcal{A}} \pi(a|x) = 1$. Such a normalization can be achieved using a softmax layer.

A softmax layer normalizes arbitrary score functions $s^a_\theta(x)$ according to

$$\pi(a|x) = \frac{\exp(s^a_\theta(x))}{\sum_{b\in\mathcal{A}} \exp(s^b_\theta(x))}\,.$$

The goal in reinforcement learning is to maximize the expected reward. Supposing that contexts are drawn from a distribution $\mu \in \Delta(\mathcal{X})$, and given a parametric policy π_θ,

$$J(\theta) = \mathbb{E}_{x\sim\mu}[\mathbb{E}_{a\sim\pi_\theta(x)}[\mathbb{E}_{r\sim R(x,a)}[r]]]$$

where the above expression uses the dependence structure imposed by the causal order illustrated in Figure 1.2. The objective J writes the expected reward as a function of the policy parameters, suppressing the implicit dependence on contexts, actions, and details of the policy. This objective is our guiding quantity; we want to make it large.

Comparison with supervised learning Recall that in the setting of supervised machine learning, features x are used to predict labels y with data coming from some distribution $x, y \sim \mathcal{D}$. Given a loss

function ℓ, the key objective is find a parametric model f_θ with small expected loss

$$\mathcal{L}(\theta) = \mathbb{E}_{x,y\sim\mathcal{D}}[\ell(f_\theta(x), y)]\,.$$

How does this compare to reinforcement learning? We can think of the loss as analogous to the negative reward, the predicted label $\hat{y} = f_\theta(x)$ as analogous to the action a, the true label y as an ideal action, and the model f_θ as analogous to the policy π_θ. In fact, these identifications are valid, and any reinforcement learning algorithm could be applied to a supervised machine learning setting where we define $r = -\ell(\pi(x), y)$.

However, there is a crucial distinction: in supervised learning, the correct label is given so the loss can be directly optimized. On the other hand, in reinforcement learning, "correct" actions are only implicitly defined as those achieving the best reward. Furthermore, the reward distribution $R(x, a)$ is generally unknown, and only samples $r \sim R(x, a)$ are observed. This *black-box* access to rewards is much less informative than knowing the full functional form. While it is possible to apply a reinforcement learning algorithm to supervised training, it amounts to throwing away what we know about loss functions and true labels. It is generally much more inefficient. On the other hand, if we do have labeled "expert" actions, supervised learning algorithms may be a good way to design policies, which we discuss further in a later chapter on *imitation learning*.

1.2 Estimation

The objective $J(\theta)$ is an expectation over unknown distributions from which we only have samples. This is not an unfamiliar problem: in supervised learning, the true objective is the expected loss over the data distribution $\mathcal{L}(\theta)$, and evaluation is done by average loss over a test data set. As long as the test data set contains samples $(x_i, y_i) \sim \mathcal{D}$ for $i = 1, ..., n$, this is an unbiased estimate, often called a Monte Carlo estimate. In expectation over the data sampling distribution, the average test loss is equal to $\mathcal{L}(\theta)$. As long as n is reasonably large, the estimate should be reasonably accurate because the variance of the estimate decreases as $1/n$.

Monte Carlo estimates are named after the Monte Carlo Casino in Monaco. This is not the only time we will encounter gambling in this text. (Metropolis, *The Beginning of the Monte Carlo Method*, 1987.)

Let's review these ideas in detail in the reinforcement learning setting. Recall that $J(\theta)$ is defined as the expected reward under $x \sim \mu$, $a \sim \pi_\theta(x)$, and $r \sim R(x, a)$. Suppose we have a dataset of this interaction process $(x_1, a_1, r_1), \ldots, (x_n, a_n, r_n)$. The Monte Carlo estimate $\hat{J}_\theta$ of $J(\theta)$ is the average reward, and it is unbiased

$$\mathbb{E}[\hat{J}_\theta] = \mathbb{E}\left[\frac{1}{n}\sum_{i=1}^{n} r_i\right] = \frac{1}{n}\sum_{i=1}^{n}\mathbb{E}[r_i] = J(\theta)\,.$$

The variance of an unbiased estimate captures how much it differs from the truth on average.

$$Var(\hat{J}_\theta) = \mathbb{E}\left[(\hat{J}_\theta - \mathbb{E}[\hat{J}_\theta])^2\right] = \mathbb{E}\left[(\hat{J}_\theta - J(\theta))^2\right] .$$

Let σ_θ^2 denote the variance of r_i under π_θ. Notice three sources of randomness contribute to the variance: variation in which contexts we see, variation in which actions our policy takes, and variation in the reward itself. Then for $Var(\hat{J}_\theta)$ we have

$$\mathbb{E}\left[\left(\frac{1}{n}\sum_{i=1}^{n}(r_i - J(\theta))\right)^2\right] = \frac{1}{n^2}\sum_{i=1}^{n}\mathbb{E}\left[(r_i - J(\theta))^2\right] = \frac{\sigma_\theta^2}{n} .$$

The average of n samples reduces the variance by a factor of n, verifying the intuition that more samples are better.

Exercise 1.1. *Show that the variance of the reward σ_θ^2 can be decomposed into the following three terms. Then, explain the meaning of each of the terms.*

$$\begin{aligned}\sigma_\theta^2 = \mathrm{Var}(r) = \mathbb{E}_{x\sim\mu, a\sim\pi_\theta(x)}\left[\mathrm{Var}_{r\sim R(x,a)}(r)\right] \\ + \mathbb{E}_{x\sim\mu}\left[\mathrm{Var}_{a\sim\pi_\theta(x)}\left(\mathbb{E}_{r\sim R(x,a)}[r]\right)\right] \\ + \mathrm{Var}_{x\sim\mu}\left(\mathbb{E}_{a\sim\pi_\theta(x), r\sim R(x,a)}[r]\right)\end{aligned}$$

Exercise 1.2. *In each of the following scenarios, how does the variance of the reward compare to the general setting? Your answer show refer to the three terms in Exercise 1.*

1. *In the movie recommendation setting (Example 1.1), there is only a single user whose context x_0 never changes.*
2. *Still in the movie recommendation setting, but now back to multiple users, the policy π_θ is as follows: use a logistic regression model to predict how likely the user is to click on each movie, then recommend the one with the highest probability.*
3. *Consider training a language model to answer mathematical word problems, where contexts x are questions, actions a are integer numerical answers, and reward r is 1 if the answer is correct and 0 otherwise.*

1.2.1 Variance Reduction with Baselines

The variance of the reward σ_θ^2 depends on the variation in contexts and the randomness of the policy, in addition to inherent uncertainty in $R(x, a)$. One method for reducing variance is to account for the variation across contexts. This variation might have a large effect on rewards, but it is not affected by our actions, so we should be able to

ignore it when designing the policy. Intuitively, given a context, it is more important to consider the increased reward due to our action, rather than the raw value of reward.

Example 1.3 (Recommendation cont.)**.** Continuing the movie example, some users barely glance at the recommendation banner while others are more open to suggestions. Variation in the user population on platform contributes to the variance of the observed clicks. By estimating a per-user "suggestibility" baseline $b(x)$, we may be able to mitigate this source of variance.

Motivated by the observation, consider subtracting a context-dependent baseline and replace the reward with the *lift* $r_i - b(x_i)$. This corresponds to a new objective

$$J_b(\theta) = \mathbb{E}_{x\sim\mu}[\mathbb{E}_{a\sim\pi_\theta(x)}[\mathbb{E}_{r\sim r(x,a)}[r - b(x)]]] = J(\theta) - \mathbb{E}_{x\sim\mu}[b(x)]$$

where the second equality holds because $b(x)$ does not depend on a or r. The remaining term $\mathbb{E}_{x\sim\mu}[b(x)]$ does not depend on θ at all; the objective $J(\theta)$ is shifted by a constant value. While $J_b(\theta)$ is the expected lift rather than expected reward, we can design a reward maximizing policy equally well by optimizing $J_b(\theta)$ instead of $J(\theta)$.

So how should we pick the baseline? By the law of total variance, we can write the variance of the baseline-corrected reward

$$\begin{aligned}\mathrm{Var}(r - b(x)) &= \mathbb{E}_x\left[\mathrm{Var}_{r|x}(r - b(x))\right] + \mathrm{Var}_x\left(\mathbb{E}_{r|x}[r - b(x)]\right) \\ &= \mathbb{E}_x\left[\mathrm{Var}_{r|x}(r)\right] + \mathrm{Var}_x\left(\mathbb{E}_{r|x}[r] - b(x)\right)\end{aligned}$$

The second line follows since $b(x)$ is a constant when conditioned on x. If $b(x)$ is close to $\mathbb{E}_{a\sim\pi_\theta(x),r\sim R(x,a)}[r]$, then the second term becomes small, and the context-driven variance can be approximately eliminated. This means that a good baseline approximates the per-context expected reward. What is left in the first term depends only on the variance in reward due to the random policy and reward function.

Example 1.4 (Recommendation cont.)**.** Continuing the movie example, we may train a per-user baseline $b(x)$ through supervised learning

$$\min_b \sum_{x,a,r\in\mathcal{D}} \ell(b(x), r)$$

where $\mathcal{D}$ is a large aggregated historical dataset. The baseline captures how often users have been observed to click on movies in general. The baseline is always unbiased as long as it is independent of actions. However, it does not approximate the "gold standard" $\mathbb{E}_{r|x}[r]$. (Instead, it usually corresponds to estimating $\mathbb{E}_{a\sim\pi'(x),r\sim R(x,a)}[r]$ for old policies π'.) Nonetheless such baselines are often observed to reduce variance in practice.

1.2.2 *Off-Policy Estimation*

So far, our estimates of the objective $J(\theta)$ require that data is collected using the policy π_θ. But what if data was generated by some other policy $\pi_{\theta'}$? We call such a setting *off-policy*. The average reward is equal to $J(\theta')$ in expectation instead of $J(\theta)$. It turns out that we can still estimate $J(\theta)$ using a simple reweighting trick that we will return to several times. We first state it generally.

Fact 1.1. *Suppose that ρ, ρ' are distributions, and $u \sim \rho'$. As long as $\rho'(u) > 0$ whenever $\rho(u) > 0$,*

$$\mathbb{E}_{u\sim\rho'}\left[\frac{\rho(u)}{\rho'(u)}u\right] = \int_u \frac{\rho(u)}{\rho'(u)}u\rho'(u)du = \mathbb{E}_{u\sim\rho}[u]$$

The ratio of likelihoods is often referred to as the *importance weight* and will be denoted by w. This inspires the estimate

$$\hat{J}'_\theta = \frac{1}{n}\sum_{i=1}^{n}\frac{\pi_\theta(a_i|x_i)}{\pi_{\theta'}(a_i|x_i)}r_i = \frac{1}{n}\sum_{i=1}^{n} w_i r_i.$$

By the reweighting rule,

$$\mathbb{E}_{a\sim\pi_{\theta'}(x)}\left[\mathbb{E}_{r\sim R(x,a)}\left[\frac{\pi_\theta(a|x)}{\pi_{\theta'}(a|x)}r\right]\right] = \mathbb{E}_{a\sim\pi_\theta(x)}[\mathbb{E}_{r\sim R(x,a)}[r]]$$

and therefore $\mathbb{E}[\hat{J}'_\theta] = J(\theta)$. It seems magical to get unbiased estimates even when samples are from a different action distribution. The temper the magic, there are two important observations. One is that for the importance weight to be well defined, we need that $\pi_{\theta'}(a|x) > 0$ whenever $\pi_\theta(a|x) > 0$. Our mathematical justification fails if we try to evaluate a policy π_θ that considers actions that were impossible under the data collection policy $\pi_{\theta'}$. Second, though the estimate is still unbiased, it is not the same quality as the "on-policy" estimate $\hat{J}_\theta$. Consider the variance of the reweighted rewards under the off-policy distribution $\pi_{\theta'}$,

While we are using $\pi_{\theta'}$ to denote the alternative policy, nothing in the argument requires that it actually shares a parametric form with π_θ. In fact, as long as the data collection policy satisfies the coverage condition, it can take any form, including hand-designed and rule-based policies.

$$\begin{aligned}
Var_{\theta'}(w_i r_i) &= \mathbb{E}_{\theta'}[w_i^2 r_i^2] - \mathbb{E}_{\theta'}[w_i r_i]^2 \\
&= \mathbb{E}_\theta[w_i r_i^2] - \mathbb{E}_\theta[r_i]^2 \\
&= \mathbb{E}_\theta[w_i r_i^2] - \mathbb{E}_\theta[r_i^2] + \mathbb{E}_\theta[r_i^2] - E_\theta[r_i]^2 \\
&= \mathbb{E}_\theta[(w_i - 1)r_i^2] + Var_\theta(r_i)
\end{aligned}$$

The first equality follows from the standard identity that variance is equal to the expectation of the square minus the square of the expectation. The second line follows by the reweighting trick. The third line adds and subtracts the same quantity. The final line groups terms and applies the standard variance identity again.

This expression writes a relationship between the off-policy estimate of reward and the on-policy reward. Whether the first term

is positive or negative depends on the correlation of the importance weight with the reward. It is negative, and thus the the variance is reduced, if the weight is negatively correlated with the magnitude of the reward (squared). This occurs if π_θ down-weights actions (compared to $\pi_{\theta'}$) with large observed reward. On the other hand, the first term is positive if the importance weights are positively correlated with reward. Because we are concerned maximizing reward, the second scenario is commonly encountered. In the reinforcement learning setting, we usually expect importance weighting to increase the variance of our estimates.

Why does variance matter? Suppose we are choosing between two policies, and policy A appears to be better than policy B on average, though our estimate also has much higher variance. Which policy should we choose? Policy A is promising but risky, while Policy B is safer. If we can only make the choice once, we may prefer to avoid risk and make the more conservative choice for Policy B. However, if our setting is *adaptive* and policies can be changed down the line, we may prefer the riskier bet. The former setting is called "offline reinforcement learning," in which policies must be designed with offline data alone, without new interactions. In some sense, this is similar to supervised learning, except that there are no closed-form loss functions. In this text, we focus on "online" settings where interaction is possible. The adaptive setting will be a key topic later on. In this chapter, our focus is instead on designing policies incrementally, and we consider variance insofar as it affects local improvements.

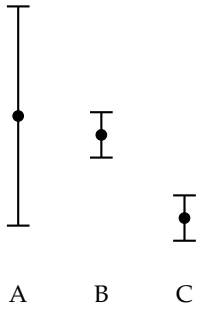


Figure 1.3: Estimated value (points) and uncertainty (intervals), which is proportional to $\sqrt{\text{variance}}$ [todo reference gaussian, heoffding, chebychev]. B is clearly preferred to C, but whether it is preferred to A is ambiguous.

Exercise 1.3. *How might you apply the off-policy reweighting idea to the baseline estimation in Example 1.4?*

1.3 Policy Gradient

So far, we studied how to estimate the expected reward of a policy given data. But how should we propose new policies? We now study ways of updating and improving policies.

1.3.1 Improvement with Gradients

Everyone knows that we train supervised models with gradients. Why are gradients useful? Because they provide a (local) improvement direction! In other words, taking a (small) step in the direction of the gradient increases the function's value. To see this, we appeal to Taylor's theorem:

$$J(\theta + u) = J(\theta) + \nabla J(\theta)^\top u + u^\top \nabla^2 J(\theta) u + \ldots$$

As long as we keep the update small, say $\|u\| \leq \varepsilon$, the higher order terms will scale as $\varepsilon^2, \varepsilon^3, \ldots$ and become negligible. Then we formalize the problem of choosing improvement step u as a constrained maximization problem:

$$\max_{\|u\| \leq \epsilon} \quad J(\theta) + \nabla J(\theta)^\top u$$

The solution is $u_\star = \alpha \nabla J(\theta)$ for $\alpha = \frac{\varepsilon}{\|\nabla J(\theta)\|}$. As long as our approximation is valid (guaranteed when α is small enough to make the higher order terms negligible), this u is an improvement step. The resulting incremental update recovers our familiar gradient ascent algorithm:

To solve the optimization problem, note that $v^\top u = \|v\|\|u\| \cos(\angle(u, v))$.

$$\theta \leftarrow \theta + \alpha \nabla J(\theta) \,.$$

We "ascend" because we are maximizing reward rather than "descending" to minimize loss.

Let's revisit our ongoing comparison with supervised learning. In supervised learning, the loss function is known, so it is possible to compute the gradient $\nabla_\theta \ell(f_\theta(x_i), y_i)$ for each data point directly. Using training data, we approximate the overall gradient of $\nabla \mathcal{L}(\theta)$ by the average gradient over the training set (or a minibatch).

Example 1.5 (Supervised learning)**.** Suppose $f_\theta(x)$ is a probabilistic model, assigning probabilities $f_\theta(y|x)$ to each possible label y, and we use the cross entropy loss $\ell(f_\theta(x), y) = -\log f_\theta(y|x)$. Using the estimated gradient yields the descent step

$$\theta \leftarrow \theta + \alpha \frac{1}{n} \sum_{i=1}^{n} \nabla \log f_\theta(y_i|x_i) \,.$$

This update encourages the model to make the observed labels y_i more likely given corresponding x_i.

1.3.2 *Gradient Estimation*

The fact of black box rewards leads to a key difference compared with supervised learning. We must find a way to estimate gradients without knowing the form of the reward function. Let let ρ_θ define the joint distribution over (x, a) defined by μ and $\pi_\theta(x)$. We begin the derivation with the reweighting trick

$$\begin{aligned}
\nabla J(\theta) &= \nabla \mathbb{E}_{x,a \sim \rho_\theta}[\mathbb{E}_{r \sim R(x,a)}[r]] \\
&= \nabla \mathbb{E}_{x,a \sim \rho_{\theta'}}\left[\frac{\rho_\theta(x,a)}{\rho_{\theta'}(x,a)} \mathbb{E}_{r \sim R(x,a)}[r]\right] \\
&= \mathbb{E}_{x,a \sim \rho_{\theta'}}\left[\frac{\nabla \rho_\theta(x,a)}{\rho_{\theta'}(x,a)} \mathbb{E}_{r \sim R(x,a)}[r]\right] \\
&= \mathbb{E}_{x,a \sim \rho_{\theta'}}\left[\frac{\rho_\theta(x,a)}{\rho_{\theta'}(x,a)} \nabla \log \rho_\theta(x,a) \mathbb{E}_{r \sim R(x,a)}[r]\right]
\end{aligned}$$

The first line is the definition of J. The second line is the reweighting trick which is valid for any distribution ρ such that $\rho(x, a) > 0$

whenever $\rho_\theta(x,a) > 0$. The third line uses the linearity of expectation to move the gradient into the expectation, since the expectation no longer depends on θ. The final line uses the log-derivative trick $\nabla f(\theta) = f(\theta)\nabla \log f(\theta)$, which will allow us to remove the dependence on the unknown context distribution μ. Since $\rho_\theta(x,a) = \mu(x)\pi_\theta(a|x)$,

The log-derivative trick follows from $\frac{d}{dx}\log x = 1/x$ and the chain rule.

$$\frac{\rho_\theta(x,a)}{\rho_{\theta'}(x,a)} = \frac{\pi_\theta(a|x)}{\pi_{\theta'}(a|x)}, \quad \nabla \log \rho_\theta(x,a) = \nabla \log \pi_\theta(a|x).$$

The gradient is therefore

$$\nabla J(\theta) = \mathbb{E}_{x,a\sim\rho_{\theta'}}\left[\frac{\pi_\theta(a|x)}{\pi_{\theta'}(a|x)}\nabla \log \pi_\theta(a|x)\,\mathbb{E}_{r\sim R(x,a)}[r]\right].$$

Written in this form as an expectation, it is straightforward to construct a Monte Carlo estimate of the gradient using samples. In the on-policy case where x_i, a_i, r_i are collected with policy π_θ this is known as the REINFORCE policy gradient,

The name is a acronym for "*RE*ward *I*ncrement = *N*onnegative *F*actor times *O*ffset *R*einforcement times *C*haracteristic *E*ligibility," (Williams, *Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning*, 1992.)

$$\theta \leftarrow \theta + \alpha\frac{1}{n}\sum_{i=1}^{n}\nabla \log \pi_\theta(a_i|x_i)\, r_i\,.$$

It is illuminating to compare this update to that of a supervised model under cross entropy loss from Example 1.5. The updates have a similar form, except that for REINFORCE the data points are weighted by the reward. This makes the policy more likely to take actions which achieved high reward.

In the general case that data collected under some other $\pi_{\theta'}$ (as long as $\pi_{\theta'} > 0$ whenever $\pi_\theta > 0$), the off-policy gradient estimate is

$$\widehat{\nabla J}'_\theta = \frac{1}{n}\sum_{i=1}^{n}\frac{\pi_\theta(a_i|x_i)}{\pi_{\theta'}(a_i|x_i)}\nabla \log \pi_\theta(a_i|x_i)\, r_i.$$

We can interpret this update as follows: increase the likelihood of actions which achieved high reward (large r_i) and were comparatively *unlikely* under the data collection policy $\pi_{\theta'}$ (large $w_i = \frac{\pi_\theta(a_i|x_i)}{\pi_{\theta'}(a_i|x_i)}$). Both aspects contribute to high variance, especially under multiple gradient updates. The first means that importance weights are usually positively correlated with observed rewards, and the second encourages π_θ into actions that are poorly covered by $\pi_{\theta'}$. As a result, it does not usually work well to take multiple off-policy gradient update steps on the same batch of data. In the next section, we will see how to fix this idea by more carefully constraining the distance between policies.

Actually, we can already propose the most common heuristic: simply discard datapoints from the gradient estimate as soon as (i) $w_i > 1+\epsilon$ for positive instances $r_i - b(x_i) > 0$ or (ii) $w_i < 1-\epsilon$ for negative instances $r_i + b(x_i) < 0$. This keeps the variance down and prevents the policy from continuing to update in low coverage directions. This algorithm is known as the "clip" variant of proximal policy optimization (PPO), which we discuss in more detail in the following section. Combining this heuristic with the variance reduction trick in Exercise 1.4 gives rise to the "group relative policy optimization" algorithm which is widely used "post-training" large language models.

Before moving on, it is worth remembering the baseline trick from Section 1.2.1. In all of the estimates, we may replace r_i with $r_i - b(x_i)$ to reduce variance without introducing bias. This is true because the baseline-adjusted objective $J_b(\theta)$ has the same gradient as the objective $J(\theta)$.

Exercise 1.4. *Consider training a language model to answer mathematical word problems, where contexts x are questions, actions a are integer numerical answers, and reward r is 1 if the answer is correct and 0 otherwise. In this setting, it is possible to sample multiple outcomes for each context, an observation which we can use to reduce variance. For each context x_i, sample a group of G actions $a_{i,1}, \dots, a_{i,G} \sim \pi_\theta(\cdot|x_i)$. and define the baseline $b(x_i) = \frac{1}{G}\sum_{j=1}^{G} r_{i,j}$.*

1. *What is $b(x_i)$ an unbiased estimate of? Hint: Recall the variance decomposition in Section 1.2.1.*
2. *How does the variance of $b(x_i)$ depend on G?*
3. *In practice, it is common to further standardize the rewards using $\sigma_i^2 = \frac{1}{G}\sum_{j=1}^{G}(r_{i,j} - b(x_i))^2$.*

$$\widehat{\nabla J}_{GRPO} = \frac{1}{nG}\sum_{i=1}^{n}\sum_{j=1}^{G}\nabla \log \pi_\theta(a_{i,j}|x_i)\frac{r_{i,j} - b(x_i)}{\sigma_i + \varepsilon}$$

Is this still an unbiased estimate of the gradient $\nabla J(\theta)$?

1.4 Trust Regions

Taking multiple policy gradient steps from one batch of data quickly runs into issues, due to high variance in off-policy gradient estimates. In this section, we attempt to remedy this issue by more carefully considering the distance between policies. In our motivation for the gradient descent algorithm, we used the Euclidean distance to define "small" updates. This is an arbitrary choice which imposes a geometry on parameter space. However, two parameter vectors θ, θ' that are close in ℓ_2 may induce very different action distributions, and vice versa.

The *KL divergence* of two distributions ρ and ρ' over a variable u is

Named for the statisticians Kullback & Leibler who defined it in their 1951 paper, which connected (then) new ideas from information theory to classical perspectives on estimation.

$$KL(\rho' \parallel \rho) = \mathbb{E}_{u\sim\rho'}\left[\log \frac{\rho'(u)}{\rho(u)}\right] .$$

Intuitively, the KL divergence quantifies the "surprise" at seeing data from ρ' when it was expected to be from ρ. The KL divergence is not a true distance because it is asymmetric and does not satisfy the triangle inequality. However, it is always non-negative and equals zero only when the two distributions coincide.

Then, we measure how different policies are by the KL divergence averaged over contexts:

$$d_{KL}(\theta', \theta) = \mathbb{E}_{x\sim\mu}[KL(\pi_{\theta'}(\cdot|x) \parallel \pi_\theta(\cdot|x))] = \mathbb{E}_{x,a\sim\rho_{\theta'}}\left[\log \frac{\pi_{\theta'}(a|x)}{\pi_\theta(a|x)}\right] .$$

This is zero when $\theta' = \theta$ and grows as the policies diverge, regardless of how the policies happen to be parameterized.

Exercise 1.5. *Consider a softmax policy over two actions* $\{0, 1\}$ *with* $\theta \in \mathbb{R}$*:* $\pi(0) = \frac{1}{1+\exp\theta}$ *and* $\pi(1) = \frac{\exp\theta}{1+\exp\theta}$.

1. *What does the policy do when* $\theta = 0$, $\theta \to -\infty$, *and* $\theta \to +\infty$*?*
2. *Consider a fixed step* $u > 0$*. For case A, the initial* $\theta = 0$ *while for case B, the initial* $\theta = \log 99$*. Clearly, the Euclidean size of the step* u *is the same whether we are in case A or care B. Compare the resulting policies for* $u = \log 1.1$ *in case A versus case B.*
3. *Compare the KL-divergence in case A versus case B.*

1.4.1 Natural Policy Gradient

We now derive an alternative update step using d_{KL}. Like before, we will consider a first order approximation to $J(\theta)$. Instead of a Euclidean norm constraint, we will use a second order (quadratic) approximation to d_{KL}. Consider the Taylor expansion of $d_{KL}(\theta, \theta + u)$ around $u = 0$:

$$d_{KL}(\theta, \theta) + \nabla_u d_{KL}(\theta, \theta + u)|_{u=0}^{\top} u + u^{\top} \nabla_u^2 d_{KL}(\theta, \theta + u) u + \ldots$$

The first term vanishes. Then noting that $u = 0$ is a minimum (since d_{KL} is always non-negative), it must be a critical point so $\nabla_u d_{KL}(\theta, \theta + u)|_{u=0} = 0$. We are left with the quadratic term, defined by the hessian

$$F(\theta) = \mathbb{E}_{x,a \sim \rho_\theta} \left[\nabla \log \pi_\theta(a|x) \nabla \log \pi_\theta(a|x)^{\top} \right],$$

which is called the *Fisher information matrix* $F(\theta)$.

Named for Ronald Fisher who defined it in *On the Mathematical Foundations of Theoretical Statistics*, 1921.

Putting this together, we have that for small u, $d_{KL}(\theta, \theta + u) \approx u^{\top} F(\theta)\, u$. TODO draw an ellipsoid We now derive an approximate KL-constrained improvement step:

$$\max_u \ \nabla J(\theta)^{\top} u \quad \text{s.t.} \quad u^{\top} F(\theta)\, u \leq \epsilon^2.$$

The solution is $u_\star = \alpha F(\theta)^{-1} \nabla J(\theta)$ for $\alpha = \epsilon (\nabla J(\theta)^{\top} F(\theta)^{-1} \nabla J(\theta))^{-1/2}$. Thus we derive the *natural policy gradient* update as

By a change of variables $v = F_0^{1/2} \Delta$, $c = F_0^{-1/2} g_0$, this reduces to $\max_{\|v\|^2 \leq \epsilon^2} c^{\top} v$, which has the same form as the previously derived gradient update step.

$$\theta \leftarrow \theta + \alpha F(\theta)^{-1} \nabla J(\theta)\,.$$

The Fisher matrix preconditions the gradient so that equal steps in parameter space correspond to equal steps in policy space. As a result, the update is less sensitive to the parameterization of π_θ.

In practice, the update must be computed from data. Given samples $x_i, a_i, r_i \sim \rho_\theta$, the gradient $\nabla J(\theta)$ is estimated with REINFORCE and the Fischer information matrix is estimated by

This algorithm was proposed by Kakade in *A Natural Policy Gradient*, 2002. A related idea is to solve the constrained optimization problem without the second order approximation (and hence without a closed-form update), explored by Schulman et al. in *Trust Region Policy Optimization*, 2015.

$$\hat{F}_\theta = \frac{1}{n}\sum_{i=1}^{n} \nabla \log \pi_\theta(a_i|x_i) \nabla \log \pi_\theta(a_i|x_i)^\top .$$

Exercise 1.6. *We will verify the first-order computation for the Taylor expansion of d_{KL}. First, show that*

$$\nabla_u d_{KL}(\theta, \theta+u) = -\mathbb{E}_{x,a\sim\rho_\theta}[\nabla_u \log \pi_{\theta+u}(a|x)] .$$

Evaluating at $u = 0$, this simplifies to $-\mathbb{E}_{x,a\sim\rho_\theta}[\nabla_\theta \log \pi_\theta(a|x)]$. Prove the following standard identity about the expected score:

$$\mathbb{E}_{x,a\sim\rho_\theta}[\nabla_\theta \log \pi_\theta(a|x)] = 0$$

Hint: you may want to use the log-derivative trick in reverse along with the fact that $\int_a \pi(a|x)da = 1$.

Exercise 1.7. *Recall the softmax policy setting from Exercise 1.5. Derive and then compare the expressions for the policy gradient update and the natural policy gradient update.*

1.4.2 Proximal Policy Optimization

While the natural policy gradient gives a principled update direction, it requires inverting estimates of the Fisher matrix $F(\theta)$, which is expensive for large deep network policies that have millions or even billions of parameters. An alternative approach replaces the KL constraint with a soft penalty. Suppose we have collected data with θ'. We frame the update step with the following optimization problem:

$$\max_\theta \; J(\theta) - \lambda\, d_{KL}(\theta', \theta) = \max_\theta J_\lambda(\theta; \theta')$$

Rather than consider first or second order approximations, we will take several gradient steps on this objective. Since the data was collected by the policy $\pi_{\theta'}$, we use the off-policy REINFORCE expression for $\nabla J(\theta)$. The gradient of the KL penalty is (see Exercise 1.6)

$$\nabla_\theta d_{KL}(\theta', \theta) = -\mathbb{E}_{x,a\sim\rho_{\theta'}}[\nabla \log \pi_\theta(a|x)].$$

Putting the two terms together and regrouping, the expression for the gradient of the penalized objective is

$$\nabla J_\lambda(\theta; \theta') = \mathbb{E}_{x,a\sim\rho_{\theta'}}\left[\nabla \log \pi_\theta(a|x) \; \left(\frac{\pi_\theta(a|x)}{\pi_{\theta'}(a|x)}\mathbb{E}_{r\sim R(x,a)}[r] + \lambda\right)\right]$$

The corresponding estimate from data (collected with policy $\pi_{\theta'}$) usually incorporates an additional reward baseline $b(x)$ to reduce variance. This is the update used in the *proximal policy optimization* algorithm, commonly referred to as PPO.

$$\widehat{\nabla} J'_{\lambda,\theta} = \frac{1}{n}\sum_{i=1}^{n} \nabla \log \pi_\theta(a_i|x_i) \; \left(\frac{\pi_\theta(a_i|x_i)}{\pi_{\theta'}(a_i|x_i)}(r_i - b(x_i)) + \lambda\right)$$

The most widely used variant of PPO, called PPO-clip, relies on a different approach. Instead, switching logic is used to prevent importance weights from leading to variance explosions. See the margin note at the end of Section 1.3.2 for further discussion (Schulman et al. *Proximal Policy Optimization Algorithms*, 2017).

Because this is an off-policy gradient estimate, we can take multiple steps on one batch of data. Notice how this expression differs from the off-policy REINFORCE estimate, which includes only the importance-weighted reward term. The penalty multiplier λ gives a uniform bonus to all (action, context) pairs in the dataset. This mitigates the variance blowup that can happen when θ is not well covered by θ'. Effectively, the bonus λ encourages the updated policy to imitate the data collection policy θ', and in doing so, maintains a small KL divergence.

1.5 *Deterministic Policies and Zeroth-Order Optimization*

So far in this chapter we have considered stochastic policies. In many settings, such as automatic feedback control, deterministic policies are widely used. These control policies are deterministic functions with only a handful of tunable parameters. For example, a *PID controller* maps an observed error signal to a continuous control action according to three scalar gains. Such controllers are used in settings ranging from heating and cooling in buildings to motor speed control in robots. PID controllers, along with other simple decision rules with a few tunable parameters, have been bread and butter in engineering practice for over a century.

In fact, randomness is not usually a desiderata for decision policies, and the expected reward is generally maximized by a deterministic policy. In reinforcement learning practice, after training a stochastic policy, it is common to deploy a deterministic version by taking the mode or mean action rather than sampling. Why not just start with a deterministic policy to begin with? Well, the randomness was essential for the algorithms we discussed so far. For a deterministic policy, $\log \pi_\theta(a|x)$ is either $-\infty$ or 0, and therefore gradients $\nabla_\theta \log \pi_\theta(a|x)$ provide very little information.

If $\pi(x) \in \arg\max_{a \in \mathcal{A}} \mathbb{E}_{r \sim R(s,a)}[r]$, then π achieves maximal reward. Ties can be broken deterministically, and hence there is always a deterministic optimal policy. We discuss the optimal policy in more detail in the following chapters.

To handle deterministic policies, we need a way to estimate how $J(\theta)$ changes with θ that does not require randomness in actions. The idea behind *zeroth-order* (also called derivative-free) optimization is to introduce randomness in parameter space instead. We may perturb the policy parameter and observe how the resulting (deterministic) policy performs. Define the Gaussian-smoothed objective

$$J_\delta(\theta) = \mathbb{E}_{v \sim \mathcal{N}(0,I)}\left[J(\theta + \delta v)\right],$$

which averages the objective over a distribution of parameters around θ, with $\delta > 0$ controlling the smoothing radius. For small δ this is a good proxy for the actual objective. It also has some benefits: $J_\delta(\theta)$ will be differentiable even when the underlying $J(\theta)$ is not, and furthermore, it is easy to estimate the gradient. To do so, we first

rewrite J_δ as an integral and use a change of variables $w = \theta + \delta v$.

$$J_\delta(\theta) = \int J(\theta + \delta v)\, p(v)\, dv = \int J(w)\, p\left(\frac{w-\theta}{\delta}\right) \delta^{-d}\, dw$$

The change of variables moves the dependence on θ into the density function. Taking the gradient,

$$\begin{aligned}\nabla_\theta J_\delta(\theta) &= \int J(w)\, \nabla_\theta \left[p\left(\frac{w-\theta}{\delta}\right)\right] \delta^{-d}\, dw \\ &= \int J(w)\, p\left(\frac{w-\theta}{\delta}\right) \nabla_\theta \log p\left(\frac{w-\theta}{\delta}\right) \delta^{-d}\, dw\end{aligned}$$

In the second line above, we use the log-derivative trick again. Recalling that p is a Gaussian density function, we compute

$$\nabla_\theta \log p\left(\frac{w-\theta}{\delta}\right) = \nabla_\theta \left(-\frac{1}{2} \left\| \frac{w-\theta}{\delta} \right\|^2 + \text{const} \right) = \frac{w-\theta}{\delta^2}\,.$$

Returning to the gradient expression and undoing the change of variables,

$$\begin{aligned}\nabla_\theta J_\delta(\theta) &= \int J(w)\, p\left(\frac{w-\theta}{\delta}\right) \frac{w-\theta}{\delta^2}\, \delta^{-d}\, dw \\ &= \int J(\theta + \delta v)\, p(v)\, \frac{v}{\delta}\, dv = \mathbb{E}_{v \sim \mathcal{N}(0,I)} \left[\frac{v}{\delta}\, J(\theta + \delta v) \right]\end{aligned}$$

This expression makes it clear how to apply the Monte Carlo estimate idea. Given n independently sampled perturbations $v_1, \ldots, v_n \sim \mathcal{N}(0, I)$, we take actions from each deterministic policy $\pi_{\theta+\delta v_i}$ for $j = 1, ..., m$ steps, observing reward r_{ij}. Then the estimate is

$$\widehat{\nabla J_\delta}(\theta) = \frac{1}{mn} \sum_{i=1}^{n} \sum_{j=1}^{m} \frac{v_i}{\delta}\, r_{ij}\,.$$

This gives us the basic *random search* algorithm: repeatedly perturb the policy parameters in random directions, weight each direction by the reward it achieved, and average.

A common refinement is to use the two-point estimate, which requires symmetric sampling. For each v_i, use both $\theta + \delta v_i$ and $\theta - \delta v_i$, obtaining rewards r_{ij}^+ and r_{ij}^-, and form

$$\widehat{\nabla J_\delta}(\theta) = \frac{1}{nm} \sum_{i=1}^{n} \sum_{i=1}^{m} \frac{v_i}{2\delta} \left(r_{ij}^+ - r_{ij}^- \right)\,.$$

This estimate reduces the variance significantly. One reason is that the reward difference may be much smaller than the magnitude of $J(\theta)$. The two point estimate is also easy to understand as an approximation of the classical definition of a directional derivative, $\nabla J(\theta)^\top v = \lim_{\delta \to 0} \frac{J(\theta+\delta v) J(\theta-\delta v)}{2\delta}$.

Random search underlies several practical algorithms which have found success in reinforcement learning. It is especially well motivated for structured policies with relatively low numbers of parameters compared to actions, especially when the number of actions is large or infinite (such as in the continuous case).

For examples, see Salimans et al., *Evolution Strategies as a Scalable Alternative to Reinforcement Learning*, and Mania et al., *Simple Random Search Provides a Competitive Approach to Reinforcement Learning*.

2
Sequential Decisions

Many decision making settings of interest have a sequential aspect, from solving puzzles to personalizing user news feeds. For such applications, the decision context is not an independent sample from some distribution, but rather an observation which depends on the sequence of actions. In this chapter we generalize our understanding of reinforcement learning to this sequential setting. We begin by just considering auto-regressive policies, which are a compact way to handle combinatorial action spaces. Then, we introduce a general notion of "state" and see how it opens up the scope of settings in which reinforcement learning can be applied. We consider interactive decision problems on indefinite time horizons, and carefully build the foundations needed for this setting.

2.1 From Auto-regressive Policies to State

In applications like recommendations and language models, the action space can grow quite large. Selecting a single recent movie (as in Example 1.1) requires choosing one from a hundred options, but selecting a ranked list of ten suggestions has *quintillions* of different possibilities. Similarly, while the number of characters is bounded, the possible combinations into words and sentences grows exponentially large. For such combinatorial action spaces, it would be unreasonable to directly consider such a large number of possibilities at once.

$100 \times 99 \times ... \times 91 \approx 8 \times 10^{18}$

A common strategy is to use an auto-regressive policy architecture: make one choice at a time, with each one considering the past choices. For personalized recommendation, this means choosing the top movie, then the next best movie of those remaining, then the next best, etc. For language models, this means generating one word (or token), then appending it to the context, then generating the next, and so on. At each step, it is only necessary to make a choice from a relatively small set of options. The only new requirement is that the

policy must take as input the choice history.

To formalize these ideas, we introduce some notation. We will write $\Pi_\theta(x)$ as the composite policy which produces the combinatorial outputs based on the observed context x, and use π_θ to denote the auto-regressive policy. This notation departs from that of the prior chapter. Denote by a_t the choice made at sequence position t, and letting H be the maximum sequence length, we denote by $a_{0:H-1}$ the full sequence, e.g. the list of movies or the generated sentence. We have that $a_{0:H-1} \sim \Pi_\theta(x)$ according to

$$\Pi(a_{0:H-1}|x) = \prod_{t=0}^{H-1} \pi(a_t|a_{0:t-1}, x)\,.$$

Aside from notation, nothing has yet changed from the prior chapter. In order to evaluate or optimize this policy, we need only translate the composite probabilities to the auto-regressive policy. For the importance weight, this boils down to

The change is π vs. Π; a_i vs. $a_{0:H-1,i}$.

$$\frac{\Pi_\theta(a_{0:H-1}|x)}{\Pi_{\theta'}(a_{0:H-1}|x)} = \prod_{t=0}^{H-1} \frac{\pi_\theta(a_t|a_{0:t-1}, x)}{\pi_{\theta'}(a_t|a_{0:t-1}, x)}\,.$$

For the score,

$$\nabla \log \Pi_\theta(a_{0:H-1}|x) = \sum_{t=0}^{H-1} \nabla \log \pi_\theta(a_t|a_{0:t-1}, x)\,.$$

With these expressions, we can directly use the policy optimization algorithms from the previous chapter.

There may be something unsatisfying about conditioning π on the entire history $a_{0:t-1}$, whose size grows with t. It seems inefficient, especially in settings where much of the history may be irrelevant to the decision at hand.

Example 2.1 (Maze navigation)**.** Consider using an auto-regressive policy to solve a maze: at each step, the policy observes the full sequence of past moves (up, down, left, right) and the initial map, and decides the next move. Alternatively, we could track the agent's current position on the two-dimensional grid, and choose the next move based on that position alone.

In principle, the auto-regressive policy can do just as well because the position can always be reconstructed from the initial map and the action sequence. However, it is the agent's position, not the path, that matters in a maze. The position is a far more natural and lower-dimensional summary.

Many problems of interest admit a summary like position in the maze example – quantities which capture everything relevant to future decisions, potentially without retaining the full history. We call such a summary the *state*, and denote it s_t. A state always exists, in the trivial sense that we may take $(a_{0:t-1}, x)$ as the state. Whether the state is a true compression depends on the setting.

Once an action is taken, the state may change. For example, in a maze, the position will update left, right, up, or down depending on the selected action. In the auto-regressive case, the new action is appended $(a_{0:t-1}, x) \to (a_{0:t}, x)$. Notice that in both examples, the next state depends only on the current state and action. This dependence will be a key assumption we make about how states *transition*. What happens when there might be nondeterministic transitions?

Example 2.2 (Maze with stochastic transitions)**.** Suppose moves in a maze are unreliable: heading left succeeds only some of the time, and otherwise the agent stays put. Some positions may be "slippery", causing the agent to sometimes move down regardless of the action. A combinatorial policy $\Pi(a_{0:T}|x)$ commits to a full move sequence in advance, without observing outcomes. If it is not possible to observe position, then perhaps this is the best we can do. However, if position can be observed, a position-dependent policy seems preferable, since it will be able to react to where the agent actually ends up.

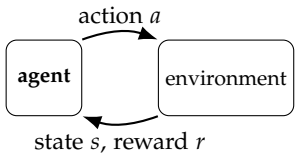


Figure 2.1: The interaction loop.

The uncertain maze example shows that the concept of state can lead to something new. Observing the state after each action and reacting to it is a substantively different setting than committing to an action sequence based on the initial context alone. This *feedback* reduces the variance of the outcome, since the policy can correct for randomness in the state transitions. This is a truly *interactive* setting, in that the agent's actions change the state of the environment, and the agent in turn observes and responds to the state it has produced.

Exercise 2.1. *For an auto-regressive policy, the off-policy gradient estimate is*

$$\frac{1}{n}\sum_{i=1}^{n}\prod_{t=0}^{H-1} w_{t,i} \sum_{t=0}^{H-1} \nabla \log \pi_\theta(a_{t,i}|a_{0:t-1,i}, x_i)\, r_i.$$

for $w_{t,i} = \frac{\pi_\theta(a_{t,i}|a_{0:t-1,i},x_i)}{\pi_{\theta'}(a_{t,i}|a_{0:t-1,i},x_i)}$*. A common approximation is to use the estimate*

$$\frac{1}{n}\sum_{i=1}^{n}\sum_{t=0}^{H-1} w_{t,i} \nabla \log \pi_\theta(a_{t,i}|a_{0:t-1,i}, x_i)\, r_i.$$

This approximation is commonly made in the PPO "clip" algorithm (and variants like GRPO) because it allows the importance weight of each point in the sequence to be decoupled.

Suppose that $|w_{t,i} - 1| \le \epsilon$*. Find an upper bound on the error of this approximation.* Hint: you may first want to use $(1-\epsilon)^m \ge 1 - m\epsilon$.

2.2 *Sequential Interaction*

Motivated by the previous section, we formalize interactive decision settings with states and transitions. States now subsume the notion of context x considered in the previous chapter.

The state of the environment is denoted s and the *state space* $\mathcal{S}$ is the set of all possible states. There is an *initial state distribution*

denoted $\mu \in \Delta(\mathcal{S})$. Actions are still denoted by $a \in \mathcal{A}$, and rewards depend on states and actions according to $r \sim R(s,a)$ for $R : \mathcal{S} \times \mathcal{A} \to \Delta(\mathbb{R})$. The environment satisfies the *Markov property* if the probability of the next state only depends on the current state and action. Mathematically,

$$\mathbb{P}(s_{t+1} \mid s_0, a_0, \ldots, s_t, a_t) = P(s_{t+1} \mid s_t, a_t)$$

where $P : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ describes the *state transitions*.

Named for the Russian mathematician Andrey Markov, who introduced this property in the early 1900s while developing theory for sequences of dependent random variables, which he later used to analyze letter patterns in the classic Russian novel *Eugene Onegin* by Alexander Pushkin.

We consider sequential settings, indexed by time steps $t = 0, 1, 2, \ldots$ and continuing indefinitely. The quantity of interest is now *cumulative* reward over these time steps. We introduce a *discount factor* $\gamma \in [0, 1]$ which multiplies future rewards as they accumulate:

$$r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k}.$$

We can think of γ as measuring how much we care about the future: if it is close to 0, we only care about near-term rewards; if it is close to 1, we put more weight on rewards far in the future. Whenever $\gamma < 1$, we can guarantee that the cumulative reward is bounded as long as $R(s,a)$ is bounded. Reviewing geometric series, you can see that when the maximum reward at any time step is one, then the maximum cumulative (discounted) reward is bounded by $\frac{1}{1-\gamma}$.

Another way to view γ is as the probability of *continuing* time step (and otherwise terminating early). If γ is close to 0, trajectories will typically be short, while if γ is close to 1, they will typically be long. As long as $\gamma < 1$, we can expect a finite time termination.

The indefinite horizon need not practically require infinite numbers of steps. Instead, termination can be modeled with an *absorbing state*, i.e. a state that transitions only to itself. For example, the end state of a maze or the solve state of a puzzle. Mathematically, we will write infinite sums, since we don't know ahead of time when we will reach the absorbing state. In practice, the interaction usually terminates at some finite time.

Putting together all of the components, this formal setting is called a *Markov decision process* (MDP), specified by the tuple

$$M = (\mathcal{S}, \mathcal{A}, \mu, P, R, \gamma).$$

The components of this tuple correspond to the properties of the environment in which our reinforcement learning agent will act.

The agent chooses actions, which in general may depend on everything up until time t. Given a policy, we repeatedly choose actions, transition states, and collect reward until termination. We call the resulting sequence of states, actions, and rewards a *trajectory*:

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots)$$

We call the interactive process which generates a trajectory *a roll-out* or *an episode*. The trajectory is a sample from a distribution induced

by μ, P, π, and R. Due to the Markov property, we mainly consider state-dependent policies $\pi : \mathcal{S} \to \Delta(\mathcal{A})$. Then, using the Markov property, the distribution ρ^π is given by

Since μ, P, r are fixed by the environment, our notation usually suppresses the dependence on them.

$$\rho^\pi(\tau) = \mu(s_0)\pi(a_0 \mid s_0)R(r_0|s_0, a_0)P(s_1 \mid s_0, a_0)\pi(a_1 \mid s_1)R(r_1|s_1, a_1)\cdots$$

We can now formalize the main goal in reinforcement learning which is to find a policy that maximizes expected cumulative reward:

$$\max_\pi \ \mathbb{E}_{\tau\sim\rho^\pi}[r_0 + \gamma r_1 + \gamma^2 r_2 \cdots] .$$

It would be natural to introduce parametrized policies, as in the previous chapter, and revisit estimation, gradients, and policy optimization in this sequential setting. In fact, we will do just that in a couple of chapters. In the present chapter, we instead develop further fundamental understanding of what sequential decisions entail.

2.3 *Exact Policy Evaluation*

Suppose we are handed the exact reward function and transition model. How can we exactly evaluate the performance of some given policy? In the previous chapter, we focused on Monte Carlo estimation, which only ever provides approximate quantities.

Towards exact evaluation, we need a way to make the infinite sum (over the indefinite horizon) tractable. Our first step is to define a state-dependent version of the cumulative reward. We define the *value function* $V^\pi : \mathcal{S} \to \mathbb{R}$ to be the expected total reward starting in a given state and following policy π:

$$V^\pi(s) = \mathbb{E}_{\tau\sim\rho^\pi}[r_0 + \gamma r_1 + \gamma^2 r_2 \cdots \mid s_0 = s] .$$

This way of defining a value function is useful because it leads to a recursive description: the cumulative reward from state s onwards is just the *current* reward plus the discounted total reward from the *next* state onwards. Mathematically,

Definition 2.1. *Bellman consistency equation for the value function*

$$V^\pi(s) = \mathbb{E}_{\substack{a\sim\pi(s) \\ s'\sim P(s,a)}} [\mathbb{E}_{r\sim R(s,a)}[r] + \gamma V^\pi(s')] \quad \forall\, s \in \mathcal{S} \tag{2.1}$$

Named after Richard Bellman who worked on decision and control theory in the context of U.S. Air Force logistics and control problems at the RAND corporation in the 1950s.

This is called the Bellman consistency equation. Recalling that expectation is a linear operation, this equation is actually a system of linear equations which characterize the value function. To see this concretely, let us restrict attention to the case of finitely many states, so that the expectation over s' can be written as an explicit

sum rather than an integral. Under this assumption, the Bellman consistency equation becomes the set of $|\mathcal{S}|$ equations

$$V^\pi(s) = \mathbb{E}_{\substack{a\sim\pi(s)\\ r\sim R(s,a)}}[r] + \gamma \sum_{s'\in\mathcal{S}} \mathbb{E}_{a\sim\pi(s)}[P(s'|s,a)]\, V^\pi(s') \quad \forall\, s \in \mathcal{S}\,,$$

where we have also simplified the expression using the linearity of expectation. Let's represent the value function as a vector V^π with dimension $|\mathcal{S}|$, and define

$$r^\pi \in \mathbb{R}^{|\mathcal{S}|}, \qquad r^\pi[s] = \mathbb{E}_{a\sim\pi(s)}\Big[\mathbb{E}_{r\sim R(s,a)}[r]\Big]\,,$$
$$P^\pi \in [0,1]^{|\mathcal{S}|\times|\mathcal{S}|}, \qquad P^\pi[s,s'] = \mathbb{E}_{a\sim\pi(s)}\big[P(s'|s,a)\big]\,.$$

Notice that by averaging the transition function $P(s'|s,a)$ over the actions $a \sim \pi(s)$, we have P^π as a transition matrix over states alone and r^π as a reward depending only on state. This state-only setting is a Markov chain: once a policy is fixed, the evolution of the state is described by the state transition matrix P^π (which is the combination of policy and environment). Different policies result in different Markov chains. Designing a good policy can be viewed as picking a good Markov chain, i.e., one that is likely to spend a lot of time in high reward states.

The resulting Markov chain is called the *closed-loop* system.

Returning to the linear Bellman consistency equation, we use the matrix-vector notation and see that it can be solved with a matrix inversion:

For now, we assume that $I - \gamma P^\pi$ is invertible.

$$V^\pi = r^\pi + \gamma P^\pi V^\pi \implies V^\pi = (I - \gamma P^\pi)^{-1} r^\pi.$$

Thus, exact policy evaluation reduces to solving a single linear system. The computational complexity of this operation depends on the size of the state and action space. We call the setting where both state space $\mathcal{S}$ and action space $\mathcal{A}$ are finite the tabular MDP setting. First, computing P^π by calculating expectations over actions requires summing over all $a \in \mathcal{A}$ for all pairs $s, s' \in \mathcal{S}$, taking $O(|\mathcal{S}|^2|\mathcal{A}|)$ time. Constructing the reward vector r^π takes $O(|\mathcal{S}||\mathcal{A}|)$ time. Finally, solving the linear system via matrix inversion takes $O(|\mathcal{S}|^3)$ time. Combining these steps, the total running time is $O(|\mathcal{S}|^3 + |\mathcal{S}|^2|\mathcal{A}|)$.

Exercise 2.2. *Assume that $\gamma < 1$. Show that $I - \gamma P^\pi$ is always invertible by showing that its nullspace is trivial.*

2.4 Iterative Policy Evaluation

When the state space grows large, a matrix inversion becomes a bottleneck. Can we trade off the requirement of finding the *exact* value function for a faster *approximate* algorithm? A simple idea for

iterative policy evaluation is to start with a guess $V_0 \in \mathbb{R}^{|\mathcal{S}|}$ and then repeatedly apply the following equation

$$V_{t+1} = r^\pi + \gamma P^\pi V_t \,.$$

Each iteration only takes $O(|\mathcal{S}|^2)$ time for the matrix-vector multiplication. One benefit of this idea is that if we happened to guess the correct V^π, the Bellman consistency equation guarantees that the update will return the same V^π unchanged. The key question, however, is whether $V_t \to V^\pi$ for an incorrect initial guess V_0.

To answer this question, define the *Bellman consistency operator* of a policy π as

$$\mathcal{B}^\pi : \mathbb{R}^{|\mathcal{S}|} \to \mathbb{R}^{|\mathcal{S}|}, \quad \mathcal{B}^\pi(V) := r^\pi + \gamma P^\pi V$$

This is a mapping which takes a $|\mathcal{S}|$ dimensional vector and returns another $|\mathcal{S}|$ dimensional vector. Our algorithm idea is simply to repeatedly apply $\mathcal{B}^\pi$. The Bellman consistency equation means that

$$V^\pi = \mathcal{B}^\pi(V^\pi),$$

or in other words, V^π is a *fixed point* of $\mathcal{B}^\pi$.

Does the Bellman consistency operator bring guesses V_t closer to the true V^π? It turns out that for any $\gamma < 1$ the answer is yes, and in fact that $\mathcal{B}^\pi$ is always a *contraction mapping*. This is a crucial property which ensures that outputs of $\mathcal{B}^\pi$ are always closer than the inputs were, for any pair of inputs. Let's measure closeness using the max norm

$$\|V\|_\infty := \sup_{s \in \mathcal{S}} |V(s)| \,.$$

Then we can show the contraction property:

$$\|\mathcal{B}^\pi(V) - \mathcal{B}^\pi(U)\|_\infty \le \gamma \|V - U\|_\infty \,.$$

We can prove this bound directly:

$$\begin{aligned}
|[\mathcal{B}^\pi(V)](s) - [\mathcal{B}^\pi(U)](s)| &= \Big| \mathop{\mathbb{E}}_{a\sim\pi(s)} \left[\mathop{\mathbb{E}}_{r\sim R(s,a)}[r] + \gamma \mathop{\mathbb{E}}_{s'\sim P(s,a)}[V(s')] \right] \\
&\qquad - \mathop{\mathbb{E}}_{a\sim\pi(s)} \left[\mathop{\mathbb{E}}_{r\sim R(s,a)}[r] + \gamma \mathop{\mathbb{E}}_{s'\sim P(s,a)}[U(s')] \right] \Big| \\
&= \gamma \left| \mathop{\mathbb{E}}_{a\sim\pi(s)} \mathop{\mathbb{E}}_{s'\sim P(s,a)} \left[V(s') - U(s') \right] \right| \\
&\le \gamma \mathop{\mathbb{E}}_{a\sim\pi(s)} \mathop{\mathbb{E}}_{s'\sim P(s,a)} \left| V(s') - U(s') \right| \qquad \text{(Jensen's inequality)} \\
&\le \gamma \max_{s'} |V(s') - U(s')| \qquad \text{(average less than maximum)} \\
&= \gamma \|V - U\|_\infty.
\end{aligned}$$

The first equality just expands the definition of $\mathcal{B}^\pi$ and in the second like terms cancel. The next line applies Jensen's inequality, moving the absolute value inside the expectation over a and s'. We then bound this expectation by the maximum value of $|V(s') - U(s')|$ over all s'. This is exactly $\|V - U\|_\infty$ by definition. Since this inequality holds for any s on the left hand side, we are done.

The contraction is true for any V, U, so we may apply it to the iterate V_t and the true value function V^π:

$$\underbrace{\|\mathcal{B}^\pi(V_t) - \mathcal{B}^\pi(V^\pi)\|_\infty}_{=\|V_{t+1} - V^\pi\|_\infty} \leq \gamma \|V_t - V^\pi\|_\infty .$$

Applying this one-step bound repeatedly, starting from the initial guess V_0, results in geometric decay of the error after t iterations:

$$\|V_t - V^\pi\|_\infty \leq \gamma^t \|V_0 - V^\pi\|_\infty.$$

How many iterations do we need for an ϵ-accurate estimate? We can work backwards to solve for t:

$$\begin{aligned} \gamma^t \|V_0 - V^\pi\|_\infty &\leq \epsilon \\ t &\geq \frac{\log(\epsilon / \|V_0 - V^\pi\|_\infty)}{\log \gamma} \\ &= \frac{\log(\|V_0 - V^\pi\|_\infty / \epsilon)}{\log(1/\gamma)}. \end{aligned}$$

Thus, the number of iterations is only logarithmic in $1/\epsilon$.

Though we have been considering a finite state space, both for concreteness and because it lets us treat V_t as an ordinary vector in $\mathbb{R}^{|\mathcal{S}|}$, the same argument can be made more generally. We can define the Bellman contraction operator $\mathcal{B}^\pi : (\mathcal{S} \to \mathbb{R}) \to (\mathcal{S} \to \mathbb{R})$ as

$$[\mathcal{B}^\pi(V)](s) := \mathbb{E}_{\substack{a \sim \pi(s) \\ s' \sim P(s,a)}} \left[\mathbb{E}_{r \sim R(s,a)}[r] + \gamma V(s') \right].$$

This is still a contraction mapping and the Bellman consistency equation still ensures that V^π is a fixed point.

The convergence argument above is in fact a special case of a much more general template that we will reuse later on. It is important enough to have a special name: the *Banach fixed-point theorem*, which states the following. Suppose X is a complete vector space equipped with a norm and $T : X \to X$ is an operator with a fixed point $x^\star$ (i.e. $T(x^\star) = x^\star$) which is a contraction, i.e. $\|T(x) - T(x')\| \leq \gamma \|x - x'\|$ for $\gamma < 1$ and all x, x'. Then, repeatedly applying T from *any* starting point converges to $x^\star$ at a geometric rate governed by γ.

Named for Polish mathematician Stefan Banach, who was a founder of functional analysis.

Exercise 2.3. *Prove the Banach fixed point theorem.*

2.5 Fixed Horizon Policy Evaluation

So far, we rely on $\gamma < 1$ to compute the value function. Without discounting, the infinite sum may not converge, the matrix may not be invertible, and the Bellman operator is not guaranteed to be a contraction. Another way to guarantee that value is well-defined and computable is to guarantee that episodes always terminate at a fixed finite time. In particular, suppose the environment terminates after exactly H time steps, for some known *horizon* $H \in \mathbb{N}$.

This is actually a special case of our more general setting, but it is worth working out how it simplifies computation of the value function. It is a special case because we may augment the state to include the time step, which deterministically increments. Since the time step is now part of this augmented state, the value function and policy may depend on t. Rather than actually use the augmented state notation, we will write V_t^π and π_t to make this dependence explicit. For simplicity we also take $\gamma = 1$ in the fixed horizon setting.

The augmented state space would be $\tilde{\mathcal{S}} = \mathcal{S} \times \{0, ..., H-1\}$ and the transitions would be defined such that $\tilde{P}(s', t' | s, t, a) = 0$ unless $t' = t + 1$. Termination at $t = H$ may simply be defined as an absorbing state.

In the fixed horizon case, the *value function* simplifies to

$$V_t^\pi(s) := \mathbb{E}_{\tau \sim \rho^\pi}[r_t + \cdots + r_{H-1} \mid s_t = s].$$

The value function still satisfies a recursive equation; the Bellman consistency equation now relates the value at time step t to the value at time step $t+1$ for all $s \in \mathcal{S},\ t = 0, \ldots, H-1$

$$V_t^\pi(s) = \mathbb{E}_{\substack{a \sim \pi_t(s) \\ s' \sim P(s,a)}}[\mathbb{E}_{r \sim R(s,a)}[r] + V_{t+1}^\pi(s')]$$

In the indefinite-horizon case, the Bellman consistency equation held for a single, time-independent value function V^π, appearing on both sides of the equation. In contrast, the fixed horizon consistency equation expresses V_t^π in terms of V_{t+1}^π. At the final time step, no reward remains to be collected, so $V_H^\pi(s) = 0$ for all s. This means we can compute the value function *exactly*: we simply start at the end of the horizon $t = H$, where the value is known, and work backwards in time, applying the Bellman consistency equation once at each time step to obtain $V_{H-1}^\pi, V_{H-2}^\pi, \ldots, V_0^\pi$ in turn. This technique is known as *dynamic programming*: solving a problem by working backwards from a known base case through a sequence of subproblems.

The term was coined by Bellman who wrote in his autobiography that he chose 'dynamic' because of the time-varying, multistage nature, but also because "it's impossible to use the word dynamic in a pejorative sense [...] something not even a Congressman could object to." Parts of the federal government were wary of funding basic mathematical research at the time.

Dynamic programming for policy evaluation is the following algorithm: First, set $V_H(s) \leftarrow 0$ for all $s \in \mathcal{S}$. Then,

- for $t = H-1, \ldots, 0$:
  - for $s \in \mathcal{S}$:
    - $V_t(s) \leftarrow \sum_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} \pi_t(a \mid s)\, P(s' \mid s, a)\, [\mathbb{E}_{r \sim R(s,a)}[r] + V_{h+1}(s')]$

Unlike the general case, there is no approximation error. This procedure computes $V_0^\pi, \ldots, V_{H-1}^\pi$ exactly, using a finite number of arithmetic operations. Updating $V_t(s)$ for a single pair (h, s) requires summing over all $a \in \mathcal{A}$ and $s' \in \mathcal{S}$. Then, counting the loops, the total running time is $O(H \cdot |\mathcal{S}|^2 \cdot |\mathcal{A}|)$: linear in the horizon, and polynomial in the sizes of the state and action spaces.

2.6 *Action-value Function*

The value function $V^\pi(s)$ is defined in terms of the reward function R and transition function P. For now, we assume these are known, and thus we can compute the value function exactly, iteratively, or using dynamic programming. Later on, we will encounter settings where R and P are *not* known in advance, and instead we must learn from data collected by interacting with the environment. In that setting, it will be useful to have a single function that packages together everything on the right-hand side of the Bellman equation. This motivates the *action-value function*, the expected total reward when starting in a given state, taking a given action, and following π thereafter:

The action-value function is also known as the Q-function

$$Q^\pi(s, a) := \mathbb{E}_{\tau \sim \rho^\pi}\left[r_0 + \gamma r_1 + \gamma^2 r_2 + \cdots \mid s_0 = s,\ a_0 = a\right]$$

Comparing this to the definition of V^π, the only difference is that the first action a is fixed, rather than drawn from $\pi(s)$. This immediately gives us a way to recover V^π from Q^π: averaging the action-value over actions drawn from the policy returns exactly the value function,

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(s)}[Q^\pi(s, a)]\,.$$

Going back to the definition of Q^π, unrolling it one time step, and then using the Markov property lets us express the action-value function in terms of the value function at the next state:

$$Q^\pi(s, a) = \mathbb{E}_{r \sim R(s,a)}[r] + \gamma\, \mathbb{E}_{s' \sim P(s,a)}[V^\pi(s')]\,.$$

The intuition is the same as before: the total reward from taking action a in state s is the immediate reward $r(s, a)$, plus the discounted value of whatever comes next. Combining the two equations, we arrive at the Bellman consistency equation for action-values.

$$Q^\pi(s, a) = \mathbb{E}_{r \sim R(s,a)}[r] + \gamma\, \mathbb{E}_{\substack{s' \sim P(s,a) \\ a' \sim \pi(s')}}[Q^\pi(s', a')]$$

Just as we did with V^π, it is possible to define an action-value Bellman consistency operator and show that it is a contraction. Similarly, we may consider the special case of fixed horizons and $\gamma = 1$. We

define the time-dependent action-value function analogously, as the expected reward-to-go from taking action a in state s at time step t:

$$\begin{aligned} Q_t^\pi(s,a) :&= \mathbb{E}_{\tau\sim\rho^\pi}[r_t + \cdots + r_{H-1} \mid s_t = s,\ a_t = a] \\ &= \mathbb{E}_{r\sim R(s,a)}[r] + \mathbb{E}_{s'\sim P(s,a)}[V_{t+1}^\pi(s')]\,. \end{aligned}$$

The value function is recovered by averaging over actions drawn from the policy at step t:

$$V_t^\pi(s) = \mathbb{E}_{a\sim\pi_t(s)}[Q_t^\pi(s,a)]\,.$$

The action-value Bellman consistency equation for fixed horizons is:

$$Q_t^\pi(s,a) = \mathbb{E}_{r\sim R(s,a)}[r] + \mathbb{E}_{\substack{s'\sim P(s,a)\\ a'\sim\pi_{t+1}(s')}}[Q_{t+1}^\pi(s',a')]$$

As with the value function, $Q_H^\pi(s,a) = 0$ for all s, a, since no reward remains to be collected after the horizon. Thus, we may apply the same backward dynamic programming procedure as for V_t^π.

The action-value function will play an important role in what follows: it is the natural object for comparing actions within a fixed state, for deriving improved policies, and for learning directly from data once R and P are no longer assumed known.

3
Optimality and Value Iteration

In the previous chapter, we saw how to evaluate a fixed policy in a Markov decision process, even when interactions unfold over long, indefinite time horizons. But evaluation is only half the story: our real goal is to *design* policies that achieve high reward. This raises a natural question: what is the right notion of *optimality* for a policy? And once we can recognize an optimal policy, how do we go about finding one?

3.1 Optimal policies

In the previous chapter, we stated the goal of reinforcement learning as finding a policy that maximizes expected cumulative reward:

$$\max_{\pi} \ \mathbb{E}_{\tau\sim\rho^{\pi}}[r_0 + \gamma r_1 + \gamma^2 r_2 + \cdots] = \max_{\pi} \ \mathbb{E}_{s\sim\mu}\left[V^{\pi}(s)\right].$$

We may now write this objective in terms of the value function by conditioning on the initial state and applying the law of total expectation. We see that the objective is the value function averaged over the initial state distribution.

Does it matter what μ is? In principle, the answer to the maximization problem could depend on it. For example, if μ places zero probability on some state s which is also never later transitioned to, then the objective is unaffected by what the policy does at s. A policy that behaves terribly there could still maximize the objective. Starting the same policy at a different initial distribution would then lead to poor performance.

This motivates us to consider a stronger notion of what it means for a policy to be optimal. The natural answer is a policy that does at least as well as every other policy, no matter which state it starts from. We call such a policy *optimal* and denote it by $\pi^\star$: for every policy π and every state $s \in \mathcal{S}$,

$$V^{\pi^\star}(s) \geq V^{\pi}(s).$$

There might be more than one optimal policy: imagine that there are two distinct actions that result in exactly the same transitions and rewards. Then it doesn't matter if the policy chooses one, the other, or both with some probability.

Since this inequality holds at every state, it continues to hold after averaging over states. So an optimal policy maximizes $\mathbb{E}_{s\sim\mu}[V^\pi(s)]$ simultaneously for *every* initial distribution μ, and we no longer need to worry about which μ we started with.

However, this definition of optimal is a demanding requirement. It is not obvious that any *single* policy beats all the others no matter the initial state. Define the *optimal value function* as the best value achievable from each state, considered separately:

$$V^\star(s) = \max_\pi V^\pi(s)\,.$$

Here, the maximizing π could in principle be different for every s. But our definition means that an optimal policy, if one exists, is able to achieve this best value at every state simultaneously, i.e. $V^{\pi^\star}(s) = V^\star(s)$ for all $s \in \mathcal{S}$.

It turns out that in every discounted MDP with finite state and action spaces, an optimal policy does exist. Moreover, it can be chosen to be state-dependent and deterministic: we never need to look further back than the current state, nor randomize over actions, to act optimally. We prove this by construction. Define the *optimal action-value function*

$$Q^\star(s,a) = \mathbb{E}_{r\sim R(s,a)}[r] + \gamma\,\mathbb{E}_{s'\sim P(s,a)}[V^\star(s')]\,.$$

Consider the *greedy policy* with respect to $Q^\star$, satisfying

$$\hat{\pi}(s) \in \arg\max_a Q^\star(s,a).$$

When the maximizing argument is unique, the policy is uniquely determined. Otherwise, we can choose any deterministic tie-breaking rule over actions.

We will show that $\hat{\pi}$ is optimal, i.e., $V^{\hat{\pi}} = V^\star$.

To show $V^{\hat{\pi}} = V^\star$, fix an arbitrary state $s \in \mathcal{S}$. By definition of $V^\star$ as the best value achievable from s, we already know $V^\star(s) \geq V^{\hat{\pi}}(s)$. So for equality to hold, we just need to show the reverse inequality, $V^\star(s) \leq V^{\hat{\pi}}(s)$. We do this in two steps: first we show that the Bellman operator $\mathcal{B}^{\hat{\pi}}$ never decreases $V^\star$, and then we use this fact, together with the contraction property from the previous chapter, to conclude that $V^\star \leq V^{\hat{\pi}}$ everywhere.

Lemma: $\mathcal{B}^{\hat{\pi}}$ never decreases $V^\star$ (elementwise):

$$[\mathcal{B}^{\hat{\pi}}(V^\star)](s) \geq V^\star(s) \qquad \forall\, s \in \mathcal{S}\,.$$

Proof: For compact notation, let $r(s,a) = \mathbb{E}_{r\sim R(s,a)}[r]$.

$$\begin{aligned}
V^\star(s) &= \max_\pi V^\pi(s) \\
&= \max_\pi \mathop{\mathbb{E}}_{a\sim\pi(s)}\left[r(s,a) + \gamma \mathop{\mathbb{E}}_{s'\sim P(s,a)} V^\pi(s')\right] && \text{Bellman consistency} \\
&\leq \max_\pi \mathop{\mathbb{E}}_{a\sim\pi(s)}\left[r(s,a) + \gamma \mathop{\mathbb{E}}_{s'\sim P(s,a)} V^\star(s')\right] && \text{definition of } V^\star
\end{aligned}$$

The inequality follows because $V^\star(s) \geq V^\pi(s)$ by definition, along with an argument that is left as an exercise. Continuing,

$$\begin{aligned} V^\star(s) &\leq \max_a \left[r(s,a) + \gamma \mathop{\mathbb{E}}_{s' \sim P(s,a)} V^\star(s') \right] && \text{depends on } \pi \text{ only via } a \sim \pi(s) \\ &= r(s, \hat{\pi}(s)) + \gamma \mathop{\mathbb{E}}_{s' \sim P(s,\hat{\pi}(s))} V^\star(s') && \text{definition of } \hat{\pi} \\ &= [\mathcal{B}^{\hat{\pi}}(V^\star)](s)\,. && \text{definition of } \mathcal{B}^{\hat{\pi}} \end{aligned}$$

Exercise 3.1. *Suppose that $V(s) \leq U(s)$ at all s. Argue why we may conclude that $\mathbb{E}_{s' \sim P(s,a)} V(s') \leq \mathbb{E}_{s' \sim P(s,a)} U(s')$.*

Before we can use this lemma, we need one more thing.
Claim: Bellman operator preserves ordering:

$$V(s) \leq U(s) \;\; \forall\, s \in \mathcal{S} \implies [\mathcal{B}^\pi(V)](s) \leq [\mathcal{B}^\pi(U)](s) \;\; \forall\, s \in \mathcal{S}\,.$$

This claim may be proved in a manner similar to Exercise 3.1. Along with the lemma, we will use it to conclude $V^\star \leq V^{\hat{\pi}}$ everywhere. Applying the claim repeatedly to the conclusion of the lemma yields an increasing chain:

We use $\leq$ between vectors to denote the pointwise (elementwise) order.

$$V^\star \leq \mathcal{B}^{\hat{\pi}}(V^\star) \leq \mathcal{B}^{\hat{\pi}}\big(\mathcal{B}^{\hat{\pi}}(V^\star)\big) \leq \cdots \leq (\mathcal{B}^{\hat{\pi}})^{(n)}(V^\star) \leq \cdots\,.$$

This is precisely the sequence of iterates produced by running iterative policy evaluation (from the previous chapter) on the policy $\hat{\pi}$, starting from the initial guess $V_0 = V^\star$. We already know that this iteration converges geometrically to its unique fixed point $V^{\hat{\pi}}$. Since $\mathcal{B}^{\hat{\pi}}$ is a γ-contraction,

$$\|(\mathcal{B}^{\hat{\pi}})^{(n)}(V^\star) - V^{\hat{\pi}}\|_\infty \leq \gamma^n \|V^\star - V^{\hat{\pi}}\|_\infty \to 0 \qquad \text{as } n \to \infty\,,$$

so in particular $(\mathcal{B}^{\hat{\pi}})^{(n)}(V^\star) \to V^{\hat{\pi}}$ pointwise.

Thus from the chain of inequalities, we can conclude $V^\star \leq V^{\hat{\pi}}$. Together with $V^\star \geq V^{\hat{\pi}}$ from the definition of $V^\star$, it must be that $V^{\hat{\pi}} = V^\star$ at every state, so $\hat{\pi}$ is indeed optimal. Therefore, an optimal policy is given by

$$\pi^\star(s) \in \arg\max_a Q^\star(s,a).$$

We have now shown that an optimal policy does exist. Furthermore, the argument shows that it can be state-dependent and deterministic, and it takes the simple form of acting greedily with respect to $Q^\star$. However, this construction is not yet an algorithm. It presupposes that we already know $Q^\star$ (or equivalently $V^\star$), when in fact this is not generally the case.

3.2 *Value Iteration*

If we can find the optimal action-value function, then we can find an optimal policy. By substituting the greedy policy into the Bellman consistency equation, we see that

$$V^\star(s) = \max_a \left[\mathbb{E}_{r\sim R(s,a)}[r] + \gamma \mathbb{E}_{s'\sim P(s,a)} V^\star(s') \right] .$$

In other words, the Bellman consistency equation for the optimal value function doesn't depend on any policy at all: it is expressed purely in terms of $V^\star$ itself. This equation is special enough to have its own name: it is called the *Bellman optimality equation*.

As we did with the consistency equation, we will assign the right hand side a special name and status as an operator. The *Bellman optimality operator* is defined as

$$[\mathcal{B}^\star(V)](s) = \max_a \left[\mathbb{E}_{r\sim R(s,a)}[r] + \gamma \mathbb{E}_{s'\sim P(s,a)} V(s') \right] .$$

The Bellman optimality equation says that $V^\star$ is a fixed point of $\mathcal{B}^\star$. Like we did in the previous chapter with $\mathcal{B}^\pi$, one can show that $\mathcal{B}^\star$ is also a γ-contraction with respect to the max norm. In fact, the proof is nearly identical. The only new step is showing that for any two functions f, g, $|\max_a f(a) - \max_a g(a)| \leq \max_a |f(a) - g(a)|$. This allows us to handle the maximum the same way expectation was handled before.

Exercise 3.2. *Let $\mathcal{A}$ be a finite set, and let $f, g : \mathcal{A} \to \mathbb{R}$ be any two functions. Show that*

$$\left| \max_a f(a) - \max_a g(a) \right| \leq \max_a |f(a) - g(a)| .$$

Exercise 3.3. *Using the exercise above, show that the Bellman optimality operator is a γ-contraction: for any two value functions V and U,*

$$\|\mathcal{B}^\star(V) - \mathcal{B}^\star(U)\|_\infty \leq \gamma \|V - U\|_\infty .$$

Thus we can conclude that repeatedly applying $\mathcal{B}^\star$ from any starting point converges to its unique fixed point $V^\star$. This gives us an algorithm in the same spirit as iterative policy evaluation: start from any initial guess and repeatedly apply $\mathcal{B}^\star$. This algorithm is known as *value iteration*.

$V_0 \leftarrow 0$
for $k = 0, 1, 2, \ldots, K-1$
    $V_{k+1} \leftarrow \mathcal{B}^\star(V_k)$
return V_K

Just like for iterative policy evaluation, after K iterations, we have $\|V_K - V^\star\|_\infty \leq \gamma^K \|V_0 - V^\star\|_\infty$.

Recall that our motivation for finding $V^\star$ was to find the policy $\pi^\star$. Therefore, the final step is to define

$$\hat{\pi}(s) \in \arg\max_a \left[\mathbb{E}_{r \sim R(s,a)}[r] + \gamma \mathbb{E}_{s' \sim P(s,a)} V_K(s') \right] .$$

This policy acts greedily with respect to the final iterate $V_K \approx V^\star$. How good is this $\hat{\pi}$ compared to the true $\pi^\star$? We answer this question by comparing the value function of this greedy policy $V^{\hat{\pi}}$ to the optimal value $V^\star$. Notice that $V^{\hat{\pi}}$ is *not* the same as V_K.

We begin by fixing a state s and comparing $V^\star(s)$ to $V^{\hat{\pi}}(s)$. Let $Q_K(s,a) = \mathbb{E}_{r \sim R(s,a)}[r] + \gamma \mathbb{E}_{s' \sim P(s,a)} V_K(s')$. Then we write an expression in terms of action-values:

$$\begin{aligned} V^\star(s) - V^{\hat{\pi}}(s) &= Q^\star(s, \pi^\star(s)) - Q^{\hat{\pi}}(s, \hat{\pi}(s)) \\ &= \left[Q^\star(s, \pi^\star(s)) - Q_K(s, \pi^\star(s)) \right] + \left[Q_K(s, \pi^\star(s)) - Q^\star(s, \hat{\pi}(s)) \right] + \left[Q^\star(s, \hat{\pi}(s)) - Q^{\hat{\pi}}(s, \hat{\pi}(s)) \right] \end{aligned}$$

For the first term we have

$$\begin{aligned} Q^\star(s, \pi^\star(s)) - Q_K(s, \pi^\star(s)) &= \gamma \, \mathbb{E}_{s' \sim P(s, \pi^\star(s))} \left[V^\star(s') - V_K(s') \right] \\ &\leq \gamma \, \mathbb{E}_{s' \sim P(s, \pi^\star(s))} \left[|V^\star(s') - V_K(s')| \right] \\ &\leq \gamma \max_{s'} |V^\star(s') - V_K(s')| = \gamma \|V^\star - V_K\|_\infty . \end{aligned}$$

For the second term we have

$$\begin{aligned} Q_K(s, \pi^\star(s)) - Q^\star(s, \hat{\pi}(s)) &\leq Q_K(s, \hat{\pi}(s)) - Q^\star(s, \hat{\pi}(s)) \\ &= \gamma \, \mathbb{E}_{s' \sim P(s, \hat{\pi}(s))} \left[V_K(s') - V^\star(s') \right] \\ &\leq \gamma \|V_K - V^\star\|_\infty \end{aligned}$$

The first inequality follows from the definition of $\hat{\pi}(s)$ and the second by similar argument as above. For the third term,

$$Q^\star(s, \hat{\pi}(s)) - Q^{\hat{\pi}}(s, \hat{\pi}(s)) = \gamma \, \mathbb{E}_{s' \sim P(s, \hat{\pi}(s))} \left[V^\star(s') - V^{\hat{\pi}}(s') \right] \leq \gamma \|V^\star - V^{\hat{\pi}}\|_\infty .$$

Putting the three terms together, and noting that the resulting bound holds for every s (and that $V^\star(s) - V^{\hat{\pi}}(s) \geq 0$), we have that

$$\|V^\star - V^{\hat{\pi}}\|_\infty \leq 2\gamma \|V_K - V^\star\|_\infty + \gamma \|V^\star - V^{\hat{\pi}}\|_\infty .$$

As a result, we conclude that

$$\|V^{\hat{\pi}} - V^\star\|_\infty \leq \frac{2\gamma}{1-\gamma} \|V_K - V^\star\|_\infty ,$$

i.e. the error in the resulting policy's value is at most a factor of $\frac{2\gamma}{1-\gamma}$ worse than the error in V_K.

Returning to the policy that results from value iteration, achieving a final policy with $\|V^{\hat{\pi}} - V^\star\|_\infty \leq \epsilon$ requires

$$\|V_K - V^\star\|_\infty \leq \frac{1-\gamma}{2\gamma}\epsilon\,,$$

meaning that we need to run value iteration for

$$K \geq \frac{\log\left(2\|V_0 - V^\star\|_\infty \gamma/(\epsilon(1-\gamma))\right)}{\log(1/\gamma)}\,.$$

3.3 *Dynamic programming*

When the time horizon is known to be fixed, the computation of the optimal policy can be done exactly, just as with policy evaluation. Recall that in the fixed-horizon setting, an episode consists of exactly H time steps, after which no further reward can be collected, and that we take $\gamma = 1$. This gives us a known base case to work backwards from. As a result, the optimal value function and optimal policy at each time step $t = 0, \ldots, H-1$ can be computed exactly, with no need for iteration or contraction arguments at all. Following this procedure backwards in time is known as **dynamic programming** (DP).

- *Base case.* At the final time step $H-1$, only one action remains, after which no further reward is collected, so the Q-function is simply the expected reward:

  $$Q^\star_{H-1}(s,a) = \mathbb{E}_{r\sim R(s,a)}[r]$$

  so the best thing to do is to pick the action that maximizes reward:

  $$\pi^\star_{H-1}(s) = \arg\max_a Q^\star_{H-1}(s,a)\,.$$

  Then $V^\star_{H-1}(s)$, the optimal value of state s at the end of the trajectory, is simply the maximum expected reward.

  $$V^\star_{H-1}(s) = \max_a Q^\star_{H-1}(s,a)$$

- *Recursion.* We work backwards in time, starting from the end, using the finite horizon Bellman equations. Specifically, for each $t = H-2, \ldots, 0$, set

  $$\begin{aligned} Q^\star_t(s,a) &= \mathbb{E}_{r\sim R(s,a)}[r] + \mathbb{E}_{s'\sim P(s,a)}[V^\star_{t+1}(s')] \\ \pi^\star_t(s) &= \arg\max_a Q^\star_t(s,a) \\ V^\star_t(s) &= \max_a Q^\star_t(s,a) \end{aligned}$$

One may also write the base case as $V^\star_H(s) = 0$ and start the iteration at $t = H-1$.

This algorithm is very similar to the policy evaluation algorithm from the previous chapter, but instead of *averaging* over the actions chosen by a policy, we instead simply take a *maximum* over the action-values. As with fixed-horizon policy evaluation, this procedure introduces no approximation error whatsoever: because we exactly know the base case $Q^\star_{H-1}$, every quantity $Q^\star_t, \pi^\star_t, V^\star_t$ is computed exactly, using a finite number of arithmetic operations. At each of the H timesteps, we must compute $Q^\star$ for each of the $|\mathcal{S}||\mathcal{A}|$ state-action pairs. Each computation takes $|\mathcal{S}|$ operations to evaluate the average value over s'. This gives a total computation time of $O(H|\mathcal{S}|^2|\mathcal{A}|)$.

The computation of the maximum is comparatively negligible at $|\mathcal{S}||\mathcal{A}|$ per time step.

3.4 *Fitted Value Iteration*

Though we have mainly worked with value functions so far, it is equally viable to work with action-value functions. Recall the definition of $Q^\star$ in terms of $V^\star$ from the previous section, and the fact that $V^\star(s') = \max_{a'} Q^\star(s', a')$. Combining these expressions, we arrive at the Bellman optimality equation for action-values:

$$Q^\star(s,a) = \mathbb{E}_{r\sim R(s,a)}[r] + \gamma\, \mathbb{E}_{s'\sim P(s,a)}\left[\max_{a'\in\mathcal{A}} Q^\star(s',a')\right].$$

We can view the right hand side of this equation as an operator $\mathcal{B}^\star_Q$ which acts on functions $Q : \mathcal{S}\times\mathcal{A}\to\mathbb{R}$. We have that $Q^\star$ is its fixed point, and an argument nearly identical to the one for $\mathcal{B}^\star$ shows that $\mathcal{B}^\star_Q$ is also a γ-contraction with respect to the max norm.

Exercise 3.4. *Prove that the operator $\mathcal{B}^\star_Q$ defined by the Bellman optimality equation for action-values is a γ-contraction with respect to the max norm.*

Therefore, repeatedly applying it from any initial guess converges to $Q^\star$. We may thus define action-value iteration by the updates

$$Q_{k+1}(s,a) = \mathbb{E}_{r\sim R(s,a)}[r] + \gamma\, \mathbb{E}_{s'\sim P(s,a)}\left[\max_{a'\in\mathcal{A}} Q_k(s',a')\right].$$

However, the exact action-value iteration, like the standard value iteration, requires computing the expectation over $r \sim R(s,a)$ and $s' \sim P(s,a)$. This can be a major practical limitation: it requires knowing the probability models of reward R and transition P. It also requires $\mathcal{S}$ and $\mathcal{A}$ to be small enough to enumerate. In many settings of interest, neither is the case. This issue motivates an alternative approach. Suppose we have one or more trajectories

States may be images, sentences, or configurations of a complex physical system.

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_{T-1}, a_{T-1}, r_{T-1}, s_T),$$

arising from some data-collection policy π_{data}, so that $a_t \sim \pi_{\text{data}}(s_t)$, $r_t \sim R(s_t, a_t)$, and $s_{t+1} \sim P(s_t, a_t)$. Each transition (s_t, a_t, r_t, s_{t+1}) in

this trajectory is a single sample from R and P. We will develop an estimate based on these samples. First, we may rewrite the Bellman update as a conditional expectation.

$$Q_{k+1}(s,a) = \mathbb{E}_{r,s'}\left[r + \gamma \max_{a'} Q_k(s',a') \mid s,a\right].$$

Then we recognize that $r_t + \gamma \max_{a'} Q_k(s_{t+1},a')$ is a single-sample Monte Carlo estimate for $Q_{k+1}(s_t,a_t)$. This is our key idea.

Since we need to fit a full action-value function, rather than just a single estimated quantity, we will move beyond Monte Carlo estimation to supervised learning. Recall that in a general setting with features x and labels y, the function f that minimizes the expected squared loss $\mathbb{E}[(y - f(x))^2]$ is the conditional expectation $f(x) = \mathbb{E}[y \mid x]$. This motivates the definition of a supervised learning problem where we use squared loss and

$$(s_t,a_t) \text{ are features}, \quad r_t + \gamma \max_{a'} Q_k(s_{t+1},a') \text{ are labels.}$$

The result is an algorithm called *Fitted Value Iteration*:

Input: dataset $\tau \sim \rho^{\pi_{\text{data}}}$, function class $\mathcal{F}$ of functions $\mathcal{S} \times \mathcal{A} \to \mathbb{R}$

Initialize $Q_0 : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$

For $k = 0, \ldots, K-1$:

$$Q_{k+1} = \arg\min_{f \in \mathcal{F}} \sum_{t=0}^{T-1} \Big(f(s_t,a_t) - \underbrace{\left(r_t + \gamma \max_a Q_k(s_{t+1},a)\right)}_{y_t} \Big)^2$$

Return $\hat{\pi}(s) = \arg\max_{a \in \mathcal{A}} Q_K(s,a) \quad \forall s$

This is a fixed-point iteration, just like ordinary value iteration, except that each application of the Bellman optimality operator is replaced by a supervised learning step on a fixed dataset. As with any supervised learning problem, if we have enough data, choose a good function class $\mathcal{F}$, and optimize well, then the learned Q_{k+1} will be a good approximation of the conditional expectation $\mathbb{E}[y \mid x]$, and thus approximately satisfy the Bellman update we wanted. However, data coverage remains important: the data only tells us about state-action pairs that π_{data} actually visits. Elsewhere, the quality of Q_{k+1} depends entirely on how the function class $\mathcal{F}$ generalizes.

What function class should we use? The simplest choice is the *tabular* class, which has one parameter per state-action pair, $f_\theta(s,a) = \theta_{s,a}$ with $\theta \in \mathbb{R}^{|\mathcal{S}||\mathcal{A}|}$. In this case the least-squares problem decouples across state-action pairs, and the solution is simply to average the labels observed for each pair. The tabular class requires $\mathcal{S}$ and $\mathcal{A}$ to be small enough to enumerate, which is one of the limitations we

Pairs that never appear in the data are unconstrained, so we must choose some default value for them, e.g. keeping the previous iterate.

hoped to avoid. Another simple option is the class of functions that are *linear in known features*, $f_\theta(s,a) = \theta^\top \phi(s,a)$, for some feature map $\phi : \mathcal{S} \times \mathcal{A} \to \mathbb{R}^d$. Now the number of parameters is d rather than $|\mathcal{S}||\mathcal{A}|$, and because state-action pairs with similar features receive similar predictions, the model can generalize to pairs that do not appear in the data. Finally, deep networks are a popular choice in practice. They have been empirically successful in many applications, though training them in this context can be unstable.

Regularized least squares has the familiar closed-form solution

$$\hat{\theta} = \Big(\sum_{t=0}^{T-1} \phi_t \phi_t^\top + \lambda I \Big)^{-1} \sum_{t=0}^{T-1} \phi_t \, y_t \,,$$

for $\phi_t = \phi(s_t, a_t)$ and regularizer $\lambda \geq 0$. (When $\lambda = 0$, this requires the matrix to be invertible.)

Exercise 3.5. *Consider the tabular function class $f_\theta(s,a) = \theta_{s,a}$.*

1. *Show that the tabular class is a special case of the linear class for a particular choice of features $\phi(s,a)$.*
2. *Let $n_{s,a}$ be the number of data points with $(s_t, a_t) = (s,a)$. Show that whenever $n_{s,a} > 0$, the least-squares solution satisfies*

$$\hat{\theta}_{s,a} = \frac{1}{n_{s,a}} \sum_{t:(s_t,a_t)=(s,a)} y_t \,.$$

A natural extension of Fitted Value Iteration is to collect additional data online. At each iteration k, we define a policy π_k from Q_k, use it to collect a fresh dataset, and use that dataset to update Q_k into Q_{k+1}. How should we choose π_k? The natural candidate is the greedy policy $\hat{\pi}_k(s) = \arg\max_{a \in \mathcal{A}} Q_k(s,a)$. However, this has zero coverage of actions that are not already favored by Q_k. Some amount of exploration is needed to keep learning about the environment. Perhaps the most common approach is the *ϵ-greedy* policy, which follows $\hat{\pi}_k$ most of the time but occasionally acts uniformly at random,

$$\pi_k(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|} & a = \hat{\pi}_k(s) \\ \frac{\epsilon}{|\mathcal{A}|} & \text{otherwise.} \end{cases}$$

Another possible approach is the *incremental* update, which slowly shifts the data-collection policy towards greedy,

$$\pi_k(a|s) = (1-\alpha)\, \pi_{k-1}(a|s) + \alpha \, \hat{\pi}(a|s) \,,$$

Then, how should we update Q_{k+1}? We may not want to solve the full empirical risk minimization problem from scratch at each iteration. Instead, it is common to update incrementally with a gradient step (or a small number of steps):

$$\theta_{k+1} = \theta_k - \alpha \nabla_\theta \sum_{t=0}^{T-1} \big(f_\theta(s_t, a_t) - y_t \big)^2 \Big|_{\theta = \theta_k}$$

The gradient offers another perspective on choosing labels:

$$\nabla_\theta \big(f_\theta(x) - y \big)^2 = 2 \big(f_\theta(x) - y \big) \nabla_\theta f_\theta(x) \,.$$

This expression is linear in the label y, so replacing y with some $\tilde{y}$ that satisfies $\mathbb{E}[\tilde{y} \mid x] = \mathbb{E}[y \mid x]$ does not change the expected gradient. In other words, *unbiased labels lead to unbiased gradients*. Of course, noisier labels lead to higher variance.

Notice that though labels $y_t = r_t + \gamma \max_a f_{\theta_k}(s_{t+1}, a)$ also depend on the parameters, we treat them as fixed labels when taking the gradient, rather than differentiating through them.

Putting the pieces together gives an online version of fitted value iteration, better known as *Q-learning:*

Initialize $Q_0 : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$

For $k = 0, \dots, K-1$:

  Define π_k from Q_k (e.g. ϵ-greedy)

  Sample a dataset $\tau \sim \rho^{\pi_k}$

  Update to Q_{k+1} with τ (e.g. gradient descent)

Return $\hat{\pi}(s) = \arg\max_{a \in \mathcal{A}} Q_K(s, a) \quad \forall s$

The name Q-learning most often refers to the particular instance described in the exercise. It was introduced by Watkins in his PhD thesis, *Learning from Delayed Rewards*, 1989.

Deep Q-learning, often called DQN, was popularized by Mnih et al., *Human-level Control through Deep Reinforcement Learning*, 2015, which learned to play Atari games from pixels.

Exercise 3.6. *Suppose that you use the tabular function class. Show that for an incremental gradient descent update from a single transition (s_t, a_t, r_t, s_{t+1}), the result is (absorbing constant factors into the step size α)*

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \Big(r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \Big) .$$

4
Iterative Policy Improvement

In the previous chapter, we developed algorithms for iteratively approximating the optimal value $V^\star$ (or $Q^\star$), and then defining a policy at the very end. In this chapter, we take a different perspective, in which the policy is a first-class object that is improved at every iteration. This leads to *policy iteration*, which alternates between evaluating the current policy and improving it. We then consider how to approximate policy iteration from data, and finally revisit the policy optimization methods from the first chapter, now in the sequential setting.

4.1 Policy Iteration

What if we improve the policy and its value function *together*? Given a policy, we can evaluate it (i.e., compute its value) using the methods from the previous chapters. Given its (action-)value function, we can define a new policy that acts greedily. Repeating these two steps gives an algorithm known as *policy iteration*.

Policy iteration was presented by Ronald Howard in his 1958 PhD thesis, *Studies in Discrete Dynamic Programing*.

Initialize $\pi_0 : \mathcal{S} \to \mathcal{A}$

For $k = 0, \dots, K-1$:

$$V^{\pi_k} \leftarrow (I - \gamma P^{\pi_k})^{-1} r^{\pi_k} \qquad \text{(policy evaluation)}$$

$$\pi_{k+1}(s) \leftarrow \arg\max_{a \in \mathcal{A}} \Big[\underbrace{r(s,a) + \gamma \, \mathbb{E}_{s' \sim P(s,a)} \big[V^{\pi_k}(s') \big]}_{Q^{\pi_k}(s,a)} \Big] \quad \forall s \qquad \text{(policy improvement)}$$

Return π_K

As in the previous chapter, we use the shorthand $r(s,a) = \mathbb{E}_{r \sim R(s,a)}[r]$.

What exactly is different between this and value iteration? It is helpful to rewrite the value iteration update by separating the maximization into its own step.

Exercise 4.1. *Given a value estimate $V_k \in \mathbb{R}^{|\mathcal{S}|}$, define the greedy policy*

$$\pi_{k+1}(s) \in \arg\max_{a \in \mathcal{A}} \left[r(s,a) + \gamma \, \mathbb{E}_{s' \sim P(s,a)} \left[V_k(s') \right] \right].$$

Using the matrix/vector notation for r^π and P^π from the previous chapter, show that the value iteration update $V_{k+1} = \mathcal{B}^\star(V_k)$ is equivalent to

$$V_{k+1} = r^{\pi_{k+1}} + \gamma P^{\pi_{k+1}} V_k \, .$$

Exercise 4.1 shows that each step of value iteration first performs policy improvement, and then performs a *single* step of iterative policy evaluation for the new policy, starting from the previous estimate V_k. Policy iteration also alternates between improvement and evaluation; the difference is in how much effort goes into evaluation. Rather than taking a single step, policy iteration evaluates each new policy completely, solving for $V^{\pi_{k+1}}$ exactly.

Performing M evaluation steps, starting from the previous estimate, is known as modified policy iteration (Puterman & Shin, *Modified Policy Iteration Algorithms for Discounted Markov Decision Problems*, 1978). Value iteration is the special case $M = 1$ and policy iteration is the limit $M \to \infty$.

Unlike value iteration, we cannot directly use contraction properties of the Bellman operator to argue about convergence. Nevertheless, we can show that it does converge at least as quickly as value iteration. There are two parts of the argument. First, we show that each new policy is at least as good as the last. Then, we use this to show that the value functions contract towards $V^\star$.

Lemma: For every k, $V^{\pi_{k+1}} \geq V^{\pi_k}$ (elementwise).

This lemma is known as the policy improvement lemma, and it means that every policy is an improvement over the last.

Proof: We begin by relating $V^{\pi_{k+1}}$ to V^{π_k}. Recall that $\mathcal{B}^{\pi_{k+1}}$ is a γ-contraction with fixed point $V^{\pi_{k+1}}$, so iterative policy evaluation converges to $V^{\pi_{k+1}}$ from *any* initial guess. Choosing the initial guess V^{π_k}, we have

$$V^{\pi_{k+1}} = \lim_{n \to \infty} \left(\mathcal{B}^{\pi_{k+1}} \right)^{(n)} (V^{\pi_k}) \, .$$

So it suffices to show that this sequence of iterates never decreases. In particular, we now argue that $\mathcal{B}^{\pi_{k+1}}(V^{\pi_k}) \geq V^{\pi_k}$. Fix a state s. Then

$$\begin{aligned}
[\mathcal{B}^{\pi_{k+1}}(V^{\pi_k})](s) &= r(s, \pi_{k+1}(s)) + \gamma \, \mathbb{E}_{s' \sim P(s, \pi_{k+1}(s))} \left[V^{\pi_k}(s') \right] && \text{definition of } \mathcal{B}^{\pi_{k+1}} \\
&= \max_{a \in \mathcal{A}} \left[r(s,a) + \gamma \, \mathbb{E}_{s' \sim P(s,a)} \left[V^{\pi_k}(s') \right] \right] && \text{definition of } \pi_{k+1} \\
&\geq r(s, \pi_k(s)) + \gamma \, \mathbb{E}_{s' \sim P(s, \pi_k(s))} \left[V^{\pi_k}(s') \right] && \text{maximum is at least value of fixed } a = \pi_k(s) \\
&= V^{\pi_k}(s) \, . && \text{Bellman consistency}
\end{aligned}$$

This holds at every state, so $V^{\pi_k} \leq \mathcal{B}^{\pi_{k+1}}(V^{\pi_k})$.

To finish the argument, we may apply $\mathcal{B}^{\pi_{k+1}}$ to both sides of this inequality (since the Bellman consistency operator preserves ordering), repeatedly, yielding an increasing chain:

$$V^{\pi_k} \leq \mathcal{B}^{\pi_{k+1}}(V^{\pi_k}) \leq \left(\mathcal{B}^{\pi_{k+1}} \right)^{(2)} (V^{\pi_k}) \leq \cdots \leq \left(\mathcal{B}^{\pi_{k+1}} \right)^{(n)} (V^{\pi_k}) \leq \cdots \, .$$

In the limit as $n \to \infty$, we conclude $V^{\pi_k} \leq V^{\pi_{k+1}}$. □

Now we use the lemma to relate successive policies to the optimal value. Fix a state s. We have

$$\begin{aligned}
V^{\pi_{k+1}}(s) &= r(s, \pi_{k+1}(s)) + \gamma \, \mathbb{E}_{s' \sim P(s, \pi_{k+1}(s))}\left[V^{\pi_{k+1}}(s')\right] && \text{Bellman consistency for } \pi_{k+1} \\
&\geq r(s, \pi_{k+1}(s)) + \gamma \, \mathbb{E}_{s' \sim P(s, \pi_{k+1}(s))}\left[V^{\pi_k}(s')\right] && \text{lemma} \\
&= \max_{a \in \mathcal{A}} \left[r(s,a) + \gamma \, \mathbb{E}_{s' \sim P(s,a)}\left[V^{\pi_k}(s')\right] \right] && \text{definition of } \pi_{k+1} \\
&= [\mathcal{B}^\star(V^{\pi_k})](s) \,. && \text{definition of } \mathcal{B}^\star
\end{aligned}$$

The second line holds because the lemma guarantees $V^{\pi_{k+1}} \geq V^{\pi_k}$ elementwise, and averaging preserves this inequality. The third line holds because π_{k+1} chooses the action which achieves the maximum.

We also know that $V^\star(s) \geq V^{\pi_{k+1}}(s)$ by the definition of $V^\star$. Therefore,

$$\begin{aligned}
0 \leq V^\star(s) - V^{\pi_{k+1}}(s) &\leq V^\star(s) - [\mathcal{B}^\star(V^{\pi_k})](s) && \text{inequality above} \\
&= [\mathcal{B}^\star(V^\star)](s) - [\mathcal{B}^\star(V^{\pi_k})](s) && \text{Bellman optimality equation} \\
&\leq \|\mathcal{B}^\star(V^\star) - \mathcal{B}^\star(V^{\pi_k})\|_\infty && \text{definition of max norm} \\
&\leq \gamma \, \|V^\star - V^{\pi_k}\|_\infty \,. && \mathcal{B}^\star \text{ is a } \gamma\text{-contraction}
\end{aligned}$$

Since the left hand side is nonnegative, it is equal to its absolute value $|V^\star(s) - V^{\pi_{k+1}}(s)|$. This bound holds at every state s, so taking the maximum over states on the left hand side,

$$\|V^\star - V^{\pi_{k+1}}\|_\infty \leq \gamma \, \|V^\star - V^{\pi_k}\|_\infty \,.$$

Thus we have shown that the value iterates contract towards the optimal value. Applying the contraction repeatedly, $\|V^\star - V^{\pi_K}\|_\infty \leq \gamma^K \|V^\star - V^{\pi_0}\|_\infty$. This is the same geometric rate as value iteration. However, there is an important difference: the bound applies directly to the value of the *policy* π_K, rather than to an intermediate V_K. There is no need for additional analysis or factor of $\frac{2\gamma}{1-\gamma}$.

In practice, policy iteration often converges much faster than this bound suggests. In fact, it always terminates exactly after finitely many iterations (see the exercise below). Along with the monotonic improvement guarantee in the lemma above, policy iteration has many attractive properties. The trade-off, however, is that each iteration is more expensive. Exact policy evaluation takes $O(|\mathcal{S}|^3 + |\mathcal{S}|^2|\mathcal{A}|)$ time, compared with $O(|\mathcal{S}|^2|\mathcal{A}|)$ for a single step of value iteration.

Exercise 4.2. *Show that if $\pi_{k+1} = \pi_k$ at some iteration, then π_k is optimal. Then, argue that policy iteration must terminate at an optimal policy after at most $|\mathcal{A}|^{|\mathcal{S}|}$ iterations.*

The improvement step requires computing expectations over the reward and transition models. As we saw with fitted value iteration, it is often more convenient to work with action-value functions directly. If we evaluate Q^{π_k} instead of V^{π_k}, then the improvement step only requires a maximum over actions:

Initialize $\pi_0 : \mathcal{S} \to \mathcal{A}$
For $k = 0, \dots, K-1$:
    Evaluate Q^{π_k}
    $\pi_{k+1}(s) \leftarrow \arg\max_{a \in \mathcal{A}} Q^{\pi_k}(s,a) \quad \forall s$
Return π_K

Of course, the reward and transition models have not disappeared: they are now needed only in the evaluation step. To evaluate Q^π, recall the Bellman consistency equation for action-values from the second chapter,

This separation will be useful later, when we replace exact evaluation with estimates from data.

$$Q^\pi(s,a) = r(s,a) + \gamma \, \mathbb{E}_{\substack{s' \sim P(s,a) \\ a' \sim \pi(s')}} \left[Q^\pi(s', a') \right] .$$

Just as for the value function, this is a system of linear equations (now $|\mathcal{S}||\mathcal{A}|$ of them), which may be solved exactly. Alternatively, we can define the operator $\mathcal{B}^\pi_Q$ given by the right hand side, and perform *iterative policy evaluation* with action-values: initialize Q_0 and repeatedly apply

$$Q_{j+1} = \mathcal{B}^\pi_Q(Q_j) , \qquad [\mathcal{B}^\pi_Q(Q)](s,a) = r(s,a) + \gamma \, \mathbb{E}_{\substack{s' \sim P(s,a) \\ a' \sim \pi(s')}} \left[Q(s', a') \right]$$

for $j = 0, \dots, M-1$. The operator $\mathcal{B}^\pi_Q$ is a γ-contraction with respect to the max norm, by an argument nearly identical to the one for $\mathcal{B}^\pi$, so these iterates converge geometrically to Q^π.

Exercise 4.3. *Prove that $\mathcal{B}^\pi_Q$ is a γ-contraction.*

Exercise 4.4. *Compare the computational cost of evaluating Q^π exactly (by solving the linear system) to that of evaluating V^π exactly and then computing $Q^\pi(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P(s,a)}[V^\pi(s')]$ for every s, a. When the model is known, which approach is cheaper?*

4.2 *Fitted Policy Iteration*

The policy evaluation step requires computing the expectation over $r \sim R(s,a)$ and $s' \sim P(s,a)$. This is a major practical limitation: it requires knowing the reward and transition models, and it requires $\mathcal{S}$ and $\mathcal{A}$ to be small enough to enumerate. We address both issues

in the same way as in the previous chapter. Suppose we have one or more trajectories

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_{T-1}, a_{T-1}, r_{T-1}, s_T),$$

arising from some data-collection policy π_{data}, so that $a_t \sim \pi_{\text{data}}(s_t)$, $r_t \sim R(s_t, a_t)$, and $s_{t+1} \sim P(s_t, a_t)$. We would like to use these samples to evaluate a (possibly different) policy π. First, we may rewrite the iterative policy evaluation update as a conditional expectation,

Recall that we may also use the notation $\tau \sim \rho^{\pi_{\text{data}}}$.

$$Q_{j+1}(s,a) = \mathbb{E}_{r,s'}\Big[r + \gamma\, \mathbb{E}_{a' \sim \pi(s')}\big[Q_j(s', a')\big] \;\Big|\; s, a\Big].$$

Just as for fitted value iteration, this motivates a supervised learning problem with squared loss, where

$$(s_t, a_t) \text{ are features}, \quad r_t + \gamma\, \mathbb{E}_{a' \sim \pi(s_{t+1})}\big[Q_j(s_{t+1}, a')\big] \text{ are labels.}$$

Notice that the expectation over a' can be computed exactly, since we know the policy π. (The label would remain unbiased if we sampled an action from π instead, though it would have higher variance.) For a deterministic policy, it is just $r_t + \gamma Q_j(s_{t+1}, \pi(s_{t+1}))$.

Fitting Q_{j+1} to these labels and repeating is known as *fitted policy evaluation*. It is a fixed-point iteration, just like iterative policy evaluation, except that each application of the Bellman consistency operator $\mathcal{B}_Q^\pi$ is replaced by a supervised learning step.

Note that the conditional expectation of the label is $[\mathcal{B}_Q^\pi(Q_j)](s_t, a_t)$, i.e. the result of one step of iterative policy evaluation applied to the current iterate Q_j. It is *not* an unbiased estimate of $Q^\pi(s_t, a_t)$. Each label is unbiased for one step of the fixed-point iteration, and the repetition of these steps can drive the iterates towards Q^π. Using the current iterate Q_j to construct the labels in this way is known as *bootstrapping*. The use of such bootstrapped labels is often referred to as temporal difference learning, so we denote these labels by y_t^{TD}.

The same is true of the labels in fitted value iteration: $r_t + \gamma \max_{a'} Q_k(s_{t+1}, a')$ is unbiased for the update $[\mathcal{B}_Q^\star(Q_k)](s_t, a_t)$, not for $Q^\star(s_t, a_t)$.

Temporal difference learning was introduced by Sutton, *Learning to Predict by the Methods of Temporal Differences*, 1988. The ideas were also present in Arthur Samuel's 1950s checkers-playing program.

Alternating fitted policy evaluation with greedy improvement gives an algorithm called *Fitted Policy Iteration*:

Input: dataset $\tau \sim \rho^{\pi_{\text{data}}}$, function class $\mathcal{F}$ of functions $\mathcal{S} \times \mathcal{A} \to \mathbb{R}$

Initialize $\pi_0 : \mathcal{S} \to \mathcal{A}$

For $k = 0, \ldots, K-1$:

Initialize Q_0 and for $j = 0, \ldots, M-1$:

$$Q_{j+1} = \operatorname*{arg\,min}_{f \in \mathcal{F}} \sum_{t=0}^{T-1} \Big(f(s_t, a_t) - \underbrace{\big(r_t + \gamma Q_j(s_{t+1}, \pi_k(s_{t+1}))\big)}_{y_t^{\text{TD}}}\Big)^2$$

$$\pi_{k+1}(s) \leftarrow \operatorname*{arg\,max}_{a \in \mathcal{A}} Q_M(s,a) \quad \forall s$$

Return π_K

Notice that the same dataset is used to evaluate every policy π_k, even though it was collected by π_{data}. This is possible because the bootstrapped labels make no assumption about which policy collected the

data. As with fitted value iteration, however, the data only tells us about state-action pairs that π_{data} actually visits; elsewhere, the quality of Q_M depends entirely on how the function class $\mathcal{F}$ generalizes.

A natural extension of Fitted Policy Iteration is to collect additional data online. When data is collected online, bootstrapping is not the only way to construct labels. An alternative is to use the observed *return* of the trajectory from time t onwards, so

This can lead to instabilities characteristic of bootstrapped methods: the policy improvement step takes a maximum over actions, which will favor whatever actions Q_M happens to overestimate. If those actions are poorly covered by the data, evaluating the new policy will result in even more unreliable estimates.

$$(s_t, a_t) \text{ are features}, \quad y_t^{\text{MC}} = \sum_{k=0}^{\infty} \gamma^k r_{t+k} \text{ are labels.}$$

The return is a Monte Carlo estimate of $Q^\pi(s_t, a_t)$ directly, with no need to bootstrap or iterate. However, this estimate is only unbiased if all of the future actions $a_{t+1}, a_{t+2}, \dots$ were chosen by π, i.e. if the data is on-policy.

Note that it also requires an indefinite sum, meaning that the data collection trajectories must run until termination.

Exercise 4.5. *Suppose the data is collected by $\pi_{data} \neq \pi$. Using the reweighting trick from the first chapter, propose a modification of y_t^{MC} which is unbiased.*

It is worth comparing the two ways to construct labels. With y_t^{MC}, each label is a direct (noisy) observation of $Q^\pi(s_t, a_t)$. We can solve a single supervised learning problem and be done. With y_t^{TD}, each label only describes one step of the fixed-point iteration, so we must solve a sequence of supervised learning problems, each with labels constructed from the previous iterate. However, the return y_t^{MC} is a sum of many random rewards, each of which depends on a random transition and a random action. The variance of a sum typically grows with the number of random terms in it, so these labels can be very noisy. In contrast, the bootstrapped label y_t^{TD} contains only one random reward and one random next state, so it has relatively low variance. For the return y_t^{MC}, every actions must have been chosen by π, so the data must be on-policy. The bootstrapped label y_t^{TD} makes no assumption about which policy collected the data, instead directly computing the expectation over $a' \sim \pi(s_{t+1})$.

The two labels are extremes of a family:

$$y_t^{(M)} = \sum_{t'=t}^{t+M-1} \gamma^{t'-t} r_{t'} + \gamma^M \mathbb{E}_{a' \sim \pi(s_{t+M})} \left[Q_j(s_{t+M}, a') \right].$$

When the intermediate actions $a_{t+1}, \dots, a_{t+M-1}$ are chosen by π, it is an unbiased estimate of M steps of iterative policy evaluation.

It is different when doing fitted policy evaluation on the value function, using labels $y_t^V = r_t + \gamma V_j(s_{t+1})$ for $V_{j+1}(s_t)$. These labels are only unbiased when $a_t \sim \pi(s_t)$, though since they only require the *single* action a_t to come from π, a single importance weight $\frac{\pi(a_t|s_t)}{\pi_{\text{data}}(a_t|s_t)}$ suffices to correct them.

Exercise 4.6. *Suppose that you use the tabular function class, take actions $a_{t+1} \sim \pi(s_{t+1})$, and update Q with gradient descent for each single transition $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$. Show that the update is (absorbing constant factors into the step size α)*

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\big(r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)\big).$$

The algorithm in this exercise is known as SARSA, after the tuple $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$ used in each update. It was studied by Rummery & Niranjan, *On-line Q-learning Using Connectionist Systems*, 1994.

What can we guarantee about fitted policy iteration? With exact evaluation, the policy improvement lemma guarantees that each new policy is at least as good as the last. To understand what happens

when the action-values are only estimated, we need an exact expression for how much better one policy is than another. This expression is stated in terms of the *advantage function*

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)\,,$$

which measures how much better the action a is than the policy's average behavior in state s. We also recall $J(\pi) = \mathbb{E}_{s\sim\mu}[V^\pi(s)]$.

Lemma (performance difference): For any two policies π and π',

$$J(\pi') - J(\pi) = \mathbb{E}_{\tau\sim\rho^{\pi'}}\Big[\sum_{t=0}^{\infty}\gamma^t A^\pi(s_t,a_t)\Big]\,.$$

This result is also due to Kakade & Langford (2002). It gives a second proof of the policy improvement lemma. If π' acts greedily with respect to Q^π, then $A^\pi(s,\pi'(s)) = \max_a Q^\pi(s,a) - V^\pi(s) \geq 0$ at every state, since $\mathbb{E}_{a\sim\pi(s)}[A^\pi(s,a)] = 0$ and a maximum is at least as large as an average. Every term in the performance difference is nonnegative, so $J(\pi') \geq J(\pi)$, and since this holds for any μ, also $V^{\pi'} \geq V^\pi$.

In words, the improvement of π' over π is the advantage (with respect to the *old* policy π) of the actions chosen by the *new* policy π', accumulated over the trajectories of the *new* policy.

The lemma highlights what it takes to guarantee improvement when we only have estimates of the action-values. In fitted policy iteration, the new policy $\hat{\pi}_{k+1}$ acts greedily with respect to an estimate $\hat{Q} \approx Q^{\pi_k}$. This is the same as acting greedily with respect to the estimated advantage $\hat{A}$, since the subtracted term depends only on the state. Since a maximum is at least as large as an average, its estimated advantage is nonnegative, $\hat{A}(s,\hat{\pi}_{k+1}(s)) \geq 0$. Writing $A^{\pi_k} = \hat{A} + (A^{\pi_k} - \hat{A})$ in the performance difference lemma,

$\hat{A}(s,a) = \hat{Q}(s,a) - \mathbb{E}_{a'\sim\pi_k(s)}[\hat{Q}(s,a')]$

$$J(\hat{\pi}_{k+1}) - J(\pi_k) \geq -\mathbb{E}_{\tau\sim\rho^{\hat{\pi}_{k+1}}}\Big[\sum_{t=0}^{\infty}\gamma^t\Big(\hat{A}(s_t,a_t) - A^{\pi_k}(s_t,a_t)\Big)\Big]\,.$$

Thus, the improvement of the new policy depends on the error in the estimated advantage, which is determined by the error in $\hat{Q}$. The new policy is guaranteed to be nearly as good as the old one only if the estimation error is small *on average over the trajectories of the new policy*. This is a problem, because $\hat{Q}$ is fit to data collected by π_k (or π_{data}), and supervised learning only guarantees accuracy on the distribution of the training data. So even if supervised learning works perfectly, the error averaged over $\rho^{\hat{\pi}_{k+1}}$ can be large when $\hat{\pi}_{k+1}$ is far from π_k. This mismatch between training and evaluation distributions is known as *distribution shift*.

An important way to mitigate distribution shift is to make the improvement step incremental. One option is to mix the current policy with the greedy one,

$$\pi_{k+1}(a \mid s) = (1-\alpha)\,\pi_k(a \mid s) + \alpha\,\hat{\pi}_{k+1}(a \mid s)\,,$$

where $\hat{\pi}_{k+1}$ is the usual greedy policy.

This idea is known as *conservative policy iteration* (Kakade & Langford, *Approximately Optimal Approximate Reinforcement Learning*, 2002).

For parametric policies π_θ, such as deep networks, we instead take a gradient step towards the greedy policy. The greedy policy is the

one that maximizes the estimated action-value at each state, so we can define an objective by averaging over the states in our data:

$$\hat{L}_k(\theta) = \sum_{t=0}^{T-1} \mathbb{E}_{a \sim \pi_\theta(s_t)} \left[\hat{Q}(s_t, a) \right],$$

where $\hat{Q} \approx Q^{\pi_{\theta_k}}$. Using the log-derivative trick,

$$\nabla \hat{L}_k(\theta) = \sum_{t=0}^{T-1} \mathbb{E}_{a \sim \pi_\theta(s_t)} \left[\nabla \log \pi_\theta(a \mid s_t)\, \hat{Q}(s_t, a) \right].$$

This expression looks familiar: it has the same form as the policy gradient from the first chapter, except contexts are replaced by states and rewards are replaced by estimated action-values. Incremental policy iteration with gradient-based improvement looks a lot like policy optimization. To better understand the connection, we next revisit policy optimization in the sequential setting.

4.3 *Policy Gradients Revisited*

In the first chapter, we optimized parametric policies π_θ in a setting where each context was drawn independently. We now extend these ideas to the sequential setting. Let $\pi_\theta : \mathcal{S} \to \Delta(\mathcal{A})$ be a parametric stochastic policy. The objective is

$$J(\theta) = \mathbb{E}_{\tau \sim \rho^{\pi_\theta}} \Big[\sum_{t=0}^{\infty} \gamma^t r_t \Big].$$

We will now discuss how to estimate the gradient of this objective.

Recall the derivation of the policy gradient in Section 1.3.2. We may follow the same steps, except with a trajectory τ in place of the pair (x, a), and the discounted sum of rewards $\sum_t \gamma^t r_t$ in place of the reward r. Using the reweighting and log-derivative trick, we have

$$\nabla J(\theta) = \mathbb{E}_{\tau \sim \rho^{\pi_\theta}} \Big[\nabla \log \rho^{\pi_\theta}(\tau) \Big(\sum_{t=0}^{\infty} \gamma^t r_t \Big) \Big].$$

To compute the score $\nabla \log \rho^{\pi_\theta}(\tau)$, recall the trajectory distribution:

$$\rho^{\pi_\theta}(\tau) = \mu(s_0)\, \pi_\theta(a_0 \mid s_0)\, R(r_0 \mid s_0, a_0)\, P(s_1 \mid s_0, a_0)\, \pi_\theta(a_1 \mid s_1) \cdots$$

Taking the logarithm turns this product into a sum, and only the policy terms depend on θ. Therefore,

$$\nabla \log \rho^{\pi_\theta}(\tau) = \sum_{t=0}^{\infty} \nabla \log \pi_\theta(a_t \mid s_t),$$

and the unknown initial distribution μ, rewards R, and transitions P drop out. Substituting the score into the gradient,

$$\nabla J(\theta) = \mathbb{E}_{\tau \sim \rho^{\pi_\theta}} \Big[\Big(\sum_{t=0}^{\infty} \nabla \log \pi_\theta(a_t \mid s_t) \Big) \Big(\sum_{t=0}^{\infty} \gamma^t r_t \Big) \Big].$$

Recall that though we write infinite sums, in practice trajectories terminate at some finite time or are truncated.

Given n trajectories sampled with π_θ, a Monte Carlo estimate of this expression gives the REINFORCE update for sequential problems.

This expression can be simplified by observing that the action at time t cannot affect rewards that were received before time t. In fact, these terms contribute nothing to the gradient in expectation, so we may drop them:

$$\nabla J(\theta) = \mathbb{E}_{\tau\sim\rho^{\pi_\theta}}\Big[\sum_{t=0}^{\infty}\Big(\nabla \log \pi_\theta(a_t \mid s_t)\sum_{t'=t}^{\infty}\gamma^{t'} r_{t'}\Big)\Big].$$

Exercise 4.7. *Show that for any $t' < t$, $\mathbb{E}_{\tau\sim\rho^{\pi_\theta}}\left[\nabla \log \pi_\theta(a_t \mid s_t)\, r_{t'}\right] = 0$.*

Although dropping these terms does not change the expectation, it does change the Monte Carlo estimate. The estimate multiplies each score $\nabla \log \pi_\theta(a_t|s_t)$ by a sum of random rewards, and as we saw in the discussion of y_t^{MC}, the variance typically grows with the length of the sum. Removing rewards which are unrelated to a_t thus helps to reduce the variance without introducing any bias.

In fact, the remaining sum is like a discounted version of y_t^{MC}: $\sum_{t'\geq t}\gamma^{t'} r_{t'} = \gamma^t \sum_{k\geq 0}\gamma^k r_{t+k}$. Therefore, conditioned on s_t and a_t, the expectation of this reward-to-go is exactly $\gamma^t Q^{\pi_\theta}(s_t, a_t)$. Applying the law of total expectation, we arrive at an expression which connects policy gradients to action-value functions:

$$\nabla J(\theta) = \mathbb{E}_{\tau\sim\rho^{\pi_\theta}}\Big[\sum_{t=0}^{\infty}\gamma^t \nabla \log \pi_\theta(a_t \mid s_t)\, Q^{\pi_\theta}(s_t, a_t)\Big].$$

Exercise 4.8. *Show that for every t and any function $f : \mathcal{S}\times\mathcal{A} \to \mathbb{R}^d$*

Assuming that the expectation is integrable.

$$\mathbb{E}_{\tau\sim\rho^{\pi_\theta}}\Big[f(s_t, a_t)\sum_{t'\geq t}\gamma^{t'} r_{t'}\Big] = \gamma^t\, \mathbb{E}_{\tau\sim\rho^{\pi_\theta}}\Big[f(s_t, a_t)\, Q^{\pi_\theta}(s_t, a_t)\Big].$$

There is one more useful equivalent expression for the gradient. It is written in terms of the *discounted state distribution*, which averages the probability of visiting each state over time,

$$d_\mu^\pi(s) = (1-\gamma)\sum_{t=0}^{\infty}\gamma^t\, \mathbb{P}_{\tau\sim\rho^\pi}\{s_t = s\}.$$

The factor $(1-\gamma)$ ensures that d_μ^π sums to one, so that it is a valid distribution over states. We can show that

$$\nabla J(\theta) = \frac{1}{1-\gamma}\,\mathbb{E}_{s\sim d_\mu^{\pi_\theta}, a\sim\pi_\theta(s)}\left[\nabla \log \pi_\theta(a \mid s)\, Q^{\pi_\theta}(s, a)\right].$$

This result is often called the *policy gradient theorem* (Sutton et al., *Policy Gradient Methods for Reinforcement Learning with Function Approximation*, 2000).

Exercise 4.9. *Show that for any bounded function $f : \mathcal{S}\times\mathcal{A} \to \mathbb{R}$,*

$$\mathbb{E}_{\tau\sim\rho^{\pi}}\Big[\sum_{t=0}^{\infty}\gamma^t f(s_t, a_t)\Big] = \frac{1}{1-\gamma}\,\mathbb{E}_{s\sim d_\mu^{\pi}, a\sim\pi(s)}\left[f(s, a)\right].$$

Compare this expression with the gradient from the first chapter. It has the same form, with contexts $x \sim \mu$ replaced by states $s \sim d_\mu^{\pi_\theta}$, and immediate rewards replaced by action-values. As long as we account for the states that the policy visits and the long-term consequences of its actions, the sequential problem looks like a one-step problem. This also connects to the previous section. The incremental improvement step of fitted policy iteration is exactly a step of policy gradient, as long as the states are distributed according to $d_\mu^{\pi_\theta}$ and the action-value estimate is accurate, $\hat{Q} = Q^{\pi_\theta}$. We may thus view policy gradient methods as an incremental form of policy iteration.

Each of the policy gradient expressions suggests a different Monte Carlo estimate. From a single trajectory τ sampled with π_θ, they are

$$\hat{g}_1 = \sum_{t=0}^{\infty} \nabla \log \pi_\theta(a_t \mid s_t) \sum_{t'=t}^{\infty} \gamma^{t'} r_{t'} \,,$$

$$\hat{g}_2 = \sum_{t=0}^{\infty} \gamma^t \nabla \log \pi_\theta(a_t \mid s_t) \, \hat{Q}(s_t, a_t) \,,$$

$$\hat{g}_3 = \frac{1}{1-\gamma} \nabla \log \pi_\theta(a_t \mid s_t) \, \hat{Q}(s_t, a_t) \,,$$

where $\hat{Q} \approx Q^{\pi_\theta}$. The first is computed directly from sampled rewards, multiplying the score by the reward-to-go. It is unbiased, but has high variance. The second replaces the reward-to-go by its conditional expectation $Q^{\pi_\theta}(s_t, a_t)$, which reduces variance by removing the randomness of future rewards. However, using some $\hat{Q}$ fit from data introduces bias to the gradient estimate. The third also requires $\hat{Q}$, and furthermore requires the state to be distributed according to $d_\mu^{\pi_\theta}$. A benefit is that it is expressed in terms of individual state-action pairs, rather than whole trajectories.

Choosing the time t with probability $(1-\gamma)\gamma^t$ is what makes the state s_t a sample from $d_\mu^{\pi_\theta}$, by definition of the discounted state distribution. In practice, it is common to instead sample uniformly from the states in a trajectory, which effectively drops the γ^t factors and introduces bias.

Beyond bias and variance, the methods also differ in their reliance on the Markovian state assumptions. The first expression only requires that actions are chosen by π_θ and that nothing else depends on θ; it works regardless of whether the policy's input is a true Markovian state. The latter two are defined in terms of the action-value function and the state distribution, which only work when the state satisfies the Markov assumption. Finally, recall the random search methods from the first chapter. They are broadly applicable, including in the sequential setting, even for deterministic policies and without any Markov assumption. These methods simply perturb the policy parameters and observe the resulting cumulative reward $\sum_t \gamma^t r_t$, and then use these two quantities to define an update direction.

Policy gradient methods can be applied to policies which take as input *any* observation o_t, e.g. a partial view of the state or a fixed-length window of recent history, by replacing $\pi_\theta(a_t \mid s_t)$ with $\pi_\theta(a_t \mid o_t)$. However, value-based methods rely on Bellman equations, which require the Markov property. A policy depending on observations may not perform as well as one depending on the true state.

Similar to the first chapter, the variance of gradient estimates can be reduced by subtracting a baseline. The key fact is that a state-dependent baseline does not change the expected gradient.

Exercise 4.10. *Let $b : \mathcal{S} \to \mathbb{R}$ be any bounded function of the state. Show that $\mathbb{E}_{a\sim\pi_\theta(s)}\left[\nabla \log \pi_\theta(a \mid s)\, b(s)\right] = 0$ for every state s.*

The exercise means that for any state-dependent baseline, we may replace $\sum_{t'=t}^{\infty} \gamma^{t'} r_{t'}$ with $\sum_{t'=t}^{\infty} \gamma^{t'} r_{t'} - b(s_t)$ or $Q^{\pi_\theta}(s_t, a_t)$ with $Q^{\pi_\theta}(s_t, a_t) - b(s_t)$ in the gradient expressions above.

Recall from Section 1.2.1 that a good baseline approximates the expected reward in each context. In the sequential setting, the analogous quantity is the expected action-value in each state, which is exactly the value function: $\mathbb{E}_{a\sim\pi(s)}[Q^\pi(s,a)] = V^\pi(s)$. Subtracting this is exactly the advantage function, so using the value function as the baseline, we recognize the advantage function

$$\nabla J(\theta) = \frac{1}{1-\gamma}\, \mathbb{E}_{s\sim d_\mu^{\pi_\theta},\, a\sim\pi_\theta(s)}\left[\nabla \log \pi_\theta(a \mid s)\, A^{\pi_\theta}(s,a)\right].$$

Weighting the score by the advantage increases the probability of actions which are better than average, and decreases the probability of those which are worse.

Of course, we do not know A^{π_θ}, so it must be estimated. The usual approach is to first fit an estimate $\hat{V} \approx V^{\pi_\theta}$ by supervised learning on the sampled trajectories. The two most common advantage estimates are then

$$\hat{A}_t = \sum_{k\geq 0} \gamma^k r_{t+k} - \hat{V}(s_t) \qquad \text{and} \qquad \hat{A}_t = r_t + \gamma \hat{V}(s_{t+1}) - \hat{V}(s_t)\,.$$

The learning problem uses states s_t as features and either the Monte Carlo labels $\sum_{k\geq 0} \gamma^k r_{t+k}$ or the bootstrapped labels $r_t + \gamma \hat{V}(s_{t+1})$, similar to the discussion in the previous section.

We can recognize the first as the return-to-go Monte Carlo labels y_t^{MC} with $\hat{V}$ subtracted as a baseline. This expression for $\hat{A}_t$ leads to unbiased gradient estimates for any baseline $\hat{V}$, but has high variance due to the sum of rewards. The second replaces $Q^{\pi_\theta}(s_t, a_t)$ with the bootstrapped estimate $r_t + \gamma \hat{V}(s_{t+1})$, i.e. the temporal difference error of $\hat{V}$. While it has lower variance, there is bias due to $\hat{V} \neq V^{\pi_\theta}$.

Policy gradient algorithms that learn a value function (the "critic") alongside a policy (the "actor") are known as *actor-critic* methods. They bear strong resemblance to fitted policy iteration, except that the policy update is only incremental. Incremental updates are helpful because they mitigate a persistent issue in reinforcement learning, namely, that new policy results in a new distribution over trajectories ρ^π and states d_μ^π.

An actor-critic algorithm follows the basic scheme for each iteration $k = 0, \ldots, K-1$:

Sample $\tau \sim \rho^{\pi_{\theta_k}}$

Update $\hat{V} \approx V^{\pi_{\theta_k}}$ using τ

Compute $\hat{A}_t$

$\theta_{k+1} \leftarrow \theta_k + \alpha \sum_t \gamma^t \nabla \log \pi_{\theta_k}(a_t \mid s_t)\, \hat{A}_t$

Using the discounted state distribution, we may rewrite the performance difference lemma as

$$J(\pi') - J(\pi) = \frac{1}{1-\gamma}\, \mathbb{E}_{\substack{s\sim d_\mu^{\pi'} \\ a\sim\pi'(s)}}\left[A^\pi(s,a)\right].$$

In other words, the improvement of a new policy is an average of the old policy's advantages A^π over the new policy's state distribution

$d_\mu^{\pi'}$. This inspires an approach to off-policy estimates: we approximate the problematic $d_\mu^{\pi'}$ by the state distribution of the old policy d_μ^{π}, and enforce that π' is close to π so that the approximation is accurate. This yields the *surrogate objective*

$$\begin{aligned} L_\pi(\pi') &= \frac{1}{1-\gamma}\,\mathbb{E}_{s\sim d_\mu^\pi,\, a\sim\pi'(s)}\big[A^\pi(s,a)\big] \\ &= \frac{1}{1-\gamma}\,\mathbb{E}_{s\sim d_\mu^\pi,\, a\sim\pi(s)}\Big[\frac{\pi'(a\mid s)}{\pi(a\mid s)}A^\pi(s,a)\Big]\,, \end{aligned}$$

where the second expression uses the reweighting trick on the actions only. The surrogate can be estimated using data collected with π alone, and the following exercise shows that it agrees with the true improvement $J(\pi') - J(\pi)$ to first order when π' is close to π.

To make this precise, Taylor expand both quantities around $\theta = \theta'$:

$$\begin{aligned} J(\theta) - J(\theta') &= \nabla J(\theta')^\top(\theta-\theta') + \\ &\qquad O(\|\theta-\theta'\|^2)\,, \\ L_{\pi_{\theta'}}(\pi_\theta) &= L_{\pi_{\theta'}}(\pi_{\theta'}) \\ &\quad + \nabla_\theta L_{\pi_{\theta'}}(\pi_\theta)\big|_{\theta=\theta'}^\top(\theta-\theta') \\ &\quad + O(\|\theta-\theta'\|^2)\,. \end{aligned}$$

Exercise 4.11. *Consider parametric policies, and let $\pi = \pi_{\theta'}$ and $\pi' = \pi_\theta$. Show that $L_{\pi_{\theta'}}(\pi_{\theta'}) = 0$ and that $\nabla_\theta L_{\pi_{\theta'}}(\pi_\theta)\big|_{\theta=\theta'} = \nabla J(\theta')$.*

The exercise shows that the constant term is zero and the linear terms are the same. So the surrogate and the true improvement differ only in the higher order terms, which are negligible when θ is close to θ'.

We may then enforce that π' stays close to π as in the first chapter: penalize the KL divergence between the old and new policies, now averaged over states $s \sim d_\mu^\pi$ rather than contexts $x \sim \mu$,

$$d_{KL}(\theta',\theta) = \mathbb{E}_{s\sim d_\mu^{\pi_{\theta'}}}\Big[KL\big(\pi_{\theta'}(\cdot|s)\;\|\;\pi_\theta(\cdot|s)\big)\Big]\,.$$

In fact, the entire trust region section of the first chapter carries over by the substitution

$$x\sim\mu \quad\longrightarrow\quad s\sim d_\mu^{\pi_{\theta'}}\,, \qquad r - b(x) \quad\longrightarrow\quad \hat{A}^{\pi_{\theta'}}(s,a)\,.$$

Under this substitution, the natural policy gradient and proximal policy optimization are defined exactly as before.

For proximal policy optimization, we collect n trajectories with the current policy $\pi_{\theta'}$ and compute advantage estimates $\hat{A}_{i,t}$ (using a value estimate $\hat{V} \approx V^{\pi_{\theta'}}$). Holding this data fixed, we then take several gradient steps on θ, starting from $\theta = \theta'$, with the update direction

$$\frac{1}{n}\sum_{i=1}^{n}\sum_t \gamma^t\,\nabla\log\pi_\theta(a_{i,t}\mid s_{i,t})\Big(w_{i,t}\,\hat{A}_{i,t} + \lambda\Big)\,, \qquad w_{i,t} = \frac{\pi_\theta(a_{i,t}\mid s_{i,t})}{\pi_{\theta'}(a_{i,t}\mid s_{i,t})}\,.$$

The importance weights are per time step, rather than products over t, because the surrogate only reweights actions (not states).

This is the reason for the approximation made by GRPO that we encountered previously for auto-regressive policies.

It is instructive to compare these expressions with the off-policy estimates from the first chapter. There, contexts were drawn from μ, so reweighting only the action was enough to estimate $J(\theta)$ *exactly* from data collected with $\pi_{\theta'}$. In the one-step setting, the surrogate is not an approximation at all, since $d_\mu^\pi = d_\mu^{\pi'} = \mu$. In the sequential setting, however, the states visited depend on the policy. An exact

off-policy estimate would also need to reweight by $\frac{\rho^{\pi}(\tau)}{\rho^{\pi'}(\tau)}$, which requires a product of importance weights over the whole history. The surrogate keeps the variance under control, but is accurate only when the two policies are close. We have now come back to where we started, with a fuller appreciation of state-of-the-art reinforcement learning methods like PPO. They never move too far from the current policy: each iteration collects data, takes small steps on a surrogate objective which is only accurate locally, and repeats. Staying close to the data-collecting policy keeps the estimates, and therefore the update directions, reliable at improving performance, at the cost of only ever making local updates.

5
Online Decision-making

In the previous chapters, we designed policies from data, mostly taking the data collection process for granted and focusing on incremental updates. However, the data available to us depends on the actions we take. If we only ever take the actions which currently look best, we may never learn that some other action is better. On the other hand, if we spend too much time trying actions which look bad, we give up reward. This is known as the *exploration-exploitation trade-off*. We touched on it briefly when we introduced ϵ-greedy data collection for Q-learning. In this chapter, we study it carefully in the simple one-step setting where the actions have no long-term consequences.

5.1 Exploration and Regret

We begin with the simplest possible case in which the context (or state) is always the same. Then the agent repeatedly chooses from a fixed set of K actions, and each action has an unknown reward distribution. For example, a local news website can choose from K different ads to display on the main page, and it receives a reward of 1 if a user clicks on the ad and 0 otherwise.

This setting is canonically known as the *multi-armed bandit*, and the actions are called *arms*. The name comes from slot machines in casinos, which are sometimes called "one-armed bandits" since they have one arm (the lever) and take money from the player.

We denote the set of actions by $\mathcal{A} = [K] = \{1, \dots, K\}$. The agent interacts with the environment for T time steps as follows.

For $t = 0, \dots, T-1$:
  Choose action $a_t \in [K]$
  Receive reward $r_t \sim R(a_t)$

As before, $R(a)$ is the reward distribution of action a, and we write $r(a) = \mathbb{E}_{r \sim R(a)}[r]$ for its mean. For simplicity, in this section, we assume that rewards are bounded in $[0, 1]$. The agent may use all of the previous actions and rewards to choose a_t. The goal is still to maximize the expected reward. Therefore, if we knew the means, the

optimal strategy would simply be to take the best action

$$a^\star \in \arg\max_{a \in [K]} r(a).$$

Since the reward means are unknown, it is necessary to learn about the actions by trying them. Each time we take an action, we observe a reward drawn from its distribution, so we can estimate the mean $r(a)$ by averaging the rewards observed from action a. The simplest strategy is to explore uniformly: try every action in turn for N rounds. Will this lead us to identify $a^\star$? To answer this question, we need to know how accurately we can estimate the mean of each action from samples. We will use the following inequality for averages of bounded random variables.

The proof of Hoeffding's inequality is beyond the scope of these notes.

Fact (Hoeffding's inequality): Let $X_1, \ldots, X_n$ be independent and identically distributed random variables with $X_i \in [0,1]$. Then for any $\delta \in (0,1)$,

$$\mathbb{P}\left(\left|\frac{1}{n}\sum_{i=1}^{n} X_i - \mathbb{E}[X_1]\right| > \sqrt{\frac{\ln(2/\delta)}{2n}}\right) \leq \delta\,.$$

The same inequality holds for any bounded random variables. If $X_i \in [c_1, c_2]$, the width is multiplied by the length of the range, $c_2 - c_1$.

In other words, with probability at least $1-\delta$, the mean $\mathbb{E}[X_1]$ lies in the interval centered at the empirical average $\frac{1}{n}\sum_{i=1}^{n} X_i$, with width $2\sqrt{\ln(2/\delta)/(2n)}$. This is a *confidence interval* for the mean. The parameter δ is a failure probability. Since the rewards are random, it is always possible that we get unlucky. Even a fair coin can come up tails many times in a row. Hoeffding's inequality says that this is unlikely, and that it becomes less likely as the number of samples n grows: for a fixed width, the failure probability δ decreases exponentially in n.

We will also need the confidence intervals of all of the actions to hold at the same time, for which we use the union bound.

Fact (union bound): For any events $A_1, \ldots, A_n$,

$$\mathbb{P}(A_1 \cup A_2 \cup \ldots \cup A_n) \leq \sum_{i=1}^{n} \mathbb{P}(A_i)\,.$$

Exercise 5.1. *Use the union bound to show that if $\mathbb{P}(A_i) \geq 1 - \frac{\delta}{n}$ for each i, then $\mathbb{P}(A_1 \cap A_2 \cap \ldots \cap A_n) \geq 1 - \delta$.*

Returning to our exploration setting, after n rounds of uniform exploration, each action has been tried n times, and we let $\hat{r}_n(a)$ denote the average of the observed rewards. Recall that we have at most N rounds, and define the confidence width

$$w_n = \sqrt{\frac{\ln(2KN/\delta)}{2n}}\,.$$

By Hoeffding's inequality and a union bound over the K actions and N rounds, with probability at least $1-\delta$,

$$|\hat{r}_n(a) - r(a)| \leq w_n \quad \text{for all } a \in [K] \text{ and } n \leq N\,.$$

In what follows, we will assume that this event holds. Our conclusions will then hold with probability at least $1-\delta$.

For each action, the true mean $r(a)$ lies in the confidence interval $[\hat{r}_n(a) - w_n,\ \hat{r}_n(a) + w_n]$ around the estimate. We call the endpoints of this interval the lower and upper confidence bounds. After n rounds, every action has been tried the same number of times, so all of the intervals have the same width (Figure 5.1). As n grows, the intervals shrink around the true means.

How do we know when to stop exploring? In Figure 5.1, action C is clearly worse than action A: the upper confidence bound of C is below the lower confidence bound of A, so the true mean of C must be below the true mean of A. However, the intervals of A and B overlap, so we cannot yet tell which of them is best. We continue to explore until it is clear which one is best, meaning that the lower confidence bound of the empirically best action is above the upper confidence bound of every other action.

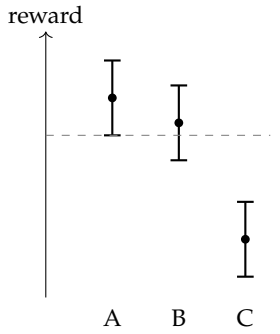


Figure 5.1: Confidence intervals after n rounds of uniform exploration, which all have the same width w_n. Action C is clearly worse than A, since its upper confidence bound is below the lower confidence bound of A (dashed line). However, the intervals of A and B still overlap, so we cannot yet tell which of them is best.

Input: $\delta \in (0,1)$, maximum number of rounds N

For $n = 1, \ldots, N$:

Try each action once, and update the averages $\hat{r}_n(a)$

$\hat{a}_n \leftarrow \arg\max_{a \in [K]} \hat{r}_n(a)$

If $\hat{r}_n(\hat{a}_n) - w_n > \hat{r}_n(a) + w_n$ for all $a \neq \hat{a}_n$: stop and return $\hat{a}_n$

When the algorithm stops, the returned action is the best action, since the confidence intervals do not overlap, so its true mean is above the true mean of every other action.

Do we stop early, and if not, what do we know after N rounds? The answer depends on how difficult it is to distinguish each action from the best one. Suppose that the best action $a^\star$ is unique, and define the *gap* of each action and the smallest gap as

$$\Delta_a = r(a^\star) - r(a)\,, \qquad \Delta = \min_{a \neq a^\star} \Delta_a\,.$$

The interval of an action a is separated from the interval of the best action once the width of both intervals is small compared with the gap Δ_a. In particular, once w_n is smaller than a quarter of the gap, the two intervals no longer overlap. So once w_n is smaller than a quarter of the *smallest* gap Δ, the interval of $a^\star$ is separated from all of the others, and we stop.

This is called a *gap-dependent* analysis, since the conclusions depend on the gaps between the means, which are unknown to the algorithm.

Exercise 5.2. *Assume that the confidence intervals hold. First, show that if $w_n < \Delta_a/4$, then $\hat{r}_n(a) + w_n < \hat{r}_n(a^\star) - w_n$. Use this to show that the algorithm stops after at most $8\ln(2KN/\delta)/\Delta^2$ rounds.*

So, does this algorithm stop? The exercise above can only guarantee stopping when $8\ln(2KN/\delta)/\Delta^2 < N$, which may not be the case if Δ is very small. The first part of this exercise tells us that every action whose interval still overlaps with that of the best action after N rounds has a gap of at most $4w_N$, so at least we can identify the near-optimal actions. If the best action is much better than the others, we identify it quickly, while if two actions have nearly the same mean, we need many samples to tell them apart. Clearly, this algorithm wastes many samples. Consider an action with a very large gap Δ_a, like action C in Figure 5.1. After a few rounds, it is clear that it is not the best action. Nonetheless, uniform exploration keeps trying it until the two *closest* actions are distinguished.

If we only care about identifying the best action eventually, these extra samples are harmless. In many applications, however, every action counts: each ad shown to a user earns a click or fails to. At each time step, the *suboptimality* of the chosen action is $r(a^\star) - r(a_t)$, which is the reward we expect to lose by not taking the best action. The cumulative suboptimality is known as the *regret*, since it measures how much we would regret not knowing the best action from the start,

$$\text{Regret}_T = \sum_{t=0}^{T-1} \big(r(a^\star) - r(a_t)\big) = \sum_{a \in [K]} N_T(a)\,\Delta_a\,.$$

Above, $N_T(a)$ is the number of times action a was taken in the first T time steps. Each time we take action a, we incur regret Δ_a, so the regret is small if we rarely take actions with large gaps. Notice that the regret is defined in terms of the mean rewards of the chosen actions. It is a random variable, since the actions depend on the random rewards. As before, we will bound it by assuming that the confidence intervals hold, so that our bounds hold with probability at least $1 - \delta$.

Exercise 5.3. *Show that when rewards are in $[0,1]$, any algorithm has $\text{Regret}_T \le T$. Also show that the algorithm which explores uniformly for all T time steps has regret $T \cdot \frac{1}{K}\sum_{a\in[K]} \Delta_a$.*

So regret which grows linearly in T is easy to achieve, even without learning anything. Our goal is to design algorithms with *sublinear* regret. Then the average regret per time step goes to zero, which means that we eventually take actions which are nearly as good as the best one.

Let's consider the regret of our uniform exploration algorithm. Suppose we can commit to the returned action once the stopping

condition holds, and take it for all of the remaining time steps (if any remain). Since there are at most T time steps available for exploration, we take $N = T/K$. Each round of exploration incurs regret $\sum_{a \neq a^\star} \Delta_a$, and after stopping, the regret is zero. So by the exercise above, with probability at least $1 - \delta$,

$$\text{Regret}_T \leq \min \left\{ \frac{8 \ln(2T/\delta)}{\Delta^2}, \frac{T}{K} \right\} \sum_{a \neq a^\star} \Delta_a \,.$$

When the smallest gap Δ is large, the regret only grows logarithmically with T. However, when Δ is small, the algorithm may not stop before time T, so the regret will grow linearly with T.

An alternative strategy is to fix the amount of exploration in advance, instead of using a stopping condition: try each action N times, and then commit to the empirically best action. This is called *explore-then-commit*. Since it does not look at the data before deciding to stop, it may commit to the wrong action. Trying each action only once leads to regret which grows linearly in T. Setting N on the order of $(T/K)^{2/3}$ leads to regret on the order of $T^{2/3}K^{1/3}$, regardless of the gaps.

Another idea is to interleave exploration and exploitation by trying a random action with some probability at every time step, and otherwise taking the action which currently looks best. This is the *ϵ-greedy* algorithm, which we encountered previously as a data collection policy for Q-learning. Swapping in ϵ-greedy exploration for uniform exploration is one way to incorporate *exploitation* earlier. However, this does not help much in the end. While the two closest actions are still being distinguished, ϵ-greedy continues to try every action in a fraction ϵ/K of the time steps, including actions which are clearly worse. The problem is that when ϵ-greedy explores, it explores uniformly.

Before the stopping condition holds, ϵ-greedy incurs regret at least $\frac{\epsilon}{K} \sum_a \Delta_a$ per time step from exploration alone. Each action is tried about $\epsilon t/K$ times by time t, so the stopping condition holds after about $8K \ln(2T/\delta)/(\epsilon\Delta^2)$ time steps. When the smallest gap Δ is small, this is larger than T, and the regret is at least $\frac{\epsilon T}{K} \sum_a \Delta_a$, which grows linearly in T. Decreasing the exploration probability over time, as $\epsilon_t \approx (K \ln t/t)^{1/3}$, leads to regret on the order of $T^{2/3}$ regardless of the gaps.

5.2 *Optimism in the Face of Uncertainty*

A more refined approach uses the confidence intervals of all of the actions to decide which ones are worth trying. One obvious idea is to stop trying an action as soon as its upper confidence bound falls below the lower confidence bound of some other action, like action C in Figure 5.1. This approach focuses on the worst actions, removing them one at a time. We will instead discuss an algorithm which focuses on the best action: at each time step, it takes the action which could plausibly be the best.

This is known as *successive elimination*. It achieves regret on the same order as UCB. We focus on UCB because its analysis is short and it extends directly to the context-dependent setting.

The *upper confidence bound* (UCB) algorithm takes the action with the highest upper confidence bound. The upper confidence bound of an action is the largest mean which is consistent with the data, in the sense that it lies in the confidence interval. So UCB follows the principle of *optimism in the face of uncertainty*: we act as if each action is as good as it could plausibly be. An action is taken either because its estimated mean is high, which is exploitation, or because its confidence interval is wide, which is exploration (Figure 5.2).

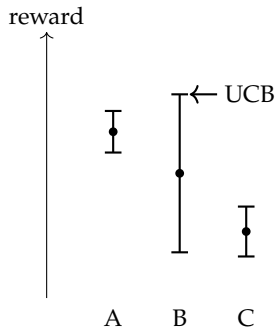


Figure 5.2: UCB takes action B, which has the highest upper confidence bound. Its estimated mean is lower than that of A, but it has been tried only a few times, so its confidence interval is wide.

Since UCB decides which action to take based on what it has seen so far, the number of times each action is taken depends on all of the previous rewards, and we need notation to keep track of it. Let $N_t(a)$

denote the number of times action a was taken before time t, and let $\hat{r}_t(a)$ denote the average of the corresponding rewards,

$$N_t(a) = \sum_{\tau=0}^{t-1} \mathbf{1}\{a_\tau = a\}\,, \qquad \hat{r}_t(a) = \frac{1}{N_t(a)} \sum_{\tau=0}^{t-1} \mathbf{1}\{a_\tau = a\}\, r_\tau\,.$$

The UCB algorithm is as follows.

Trying each action once at the start ensures that $N_t(a) \geq 1$, so the width is well defined.

Input: $\delta \in (0,1)$

Try each action once

For $t = K, \ldots, T-1$:

$$a_t \in \arg\max_{a \in [K]} \hat{r}_t(a) + w_t(a)\,, \qquad w_t(a) = \sqrt{\frac{\ln(2KT/\delta)}{2N_t(a)}}$$

Receive reward r_t, and update $\hat{r}_{t+1}(a_t)$ and $N_{t+1}(a_t)$

The confidence width $w_t(a)$ should look familiar. It is the same as the width w_n from uniform exploration, with the number of samples $N_t(a)$ in place of n, and the horizon T in place of the number of rounds N. Since different actions are now tried different numbers of times, the width depends on the action as well as the time. As before, with probability at least $1-\delta$,

$$|\hat{r}_t(a) - r(a)| \leq w_t(a) \quad \text{for all } a \in [K] \text{ and } t < T\,.$$

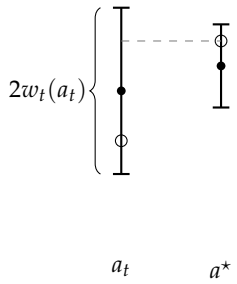


Figure 5.3: The regret of UCB at time t. Filled dots are the estimated means and open circles are the (unknown) true means. The true mean of $a^\star$ (dashed line) is below the top of its interval, which is below the top of the interval for a_t. The true mean of a_t is above the bottom of its interval. So the gap between the true means is at most the length $2w_t(a_t)$ of the interval for a_t.

We now bound the regret, assuming that the confidence intervals hold. At each time step after the first K, the regret is bounded by

$$\begin{aligned} r(a^\star) - r(a_t) &\leq \hat{r}_t(a^\star) + w_t(a^\star) - r(a_t) && \text{confidence bound for } a^\star \\ &\leq \hat{r}_t(a_t) + w_t(a_t) - r(a_t) && \text{choice of } a_t \\ &\leq 2w_t(a_t)\,. && \text{confidence bound for } a_t \end{aligned}$$

In the first line, we replace the true mean of $a^\star$ by the top of its interval, which is higher, as illustrated in Figure 5.3. In the second line, we replace the top of the interval for $a^\star$ by the top of the interval for a_t, which is higher since UCB chose a_t. In the third line, we use that the true mean of a_t is above the bottom of its interval, so the distance from the top of its interval to its true mean is at most the length of the interval, which is $2w_t(a_t)$. So the regret at each step is at most twice the width of the confidence interval of the chosen action. Summing over time,

$$\begin{aligned} \sum_{t=K}^{T-1} 2w_t(a_t) &= \sqrt{2\ln(2KT/\delta)} \sum_{t=K}^{T-1} \frac{1}{\sqrt{N_t(a_t)}} && \text{definition of } w_t \\ &= \sqrt{2\ln(2KT/\delta)} \sum_{a=1}^{K} \sum_{t \geq K:\, a_t = a} \frac{1}{\sqrt{N_t(a)}} && \text{group by action} \\ &= \sqrt{2\ln(2KT/\delta)} \sum_{a=1}^{K} \sum_{n=1}^{N_T(a)-1} \frac{1}{\sqrt{n}}\,. \end{aligned}$$

The last line is true because each time action a is taken, its count $N_t(a)$ is one larger than the previous time, starting from 1.

Exercise 5.4. *First, show that* $\sum_{n=1}^{N} \frac{1}{\sqrt{n}} \leq 2\sqrt{N}$ *by comparing the sum to an integral. Then, use the Cauchy-Schwarz inequality and* $\sum_{a=1}^{K} N_T(a) = T$ *to show that* $\sum_{a=1}^{K} \sqrt{N_T(a)} \leq \sqrt{KT}$.

Using the exercise, and adding at most K regret for the first K steps, with probability at least $1 - \delta$,

$$\text{Regret}_T \leq K + 2\sqrt{2KT \ln(2KT/\delta)}\,.$$

So the regret of UCB grows with the square root of T, up to a logarithmic factor. This does not depend on the gaps, so it holds even when the gaps are too small for uniform exploration to stop, or when the two best actions have identical means.

It turns out that in general, it is not possible to do better than $\sqrt{T}$. The intuition is that with n samples from a distribution, we can only estimate its mean to within about $1/\sqrt{n}$. If two actions have means $1/\sqrt{T}$ apart, then even with all T samples, we cannot confidently tell them apart. Thus, we choose the worse action a constant fraction of the time, leading to regret on the order of $T \cdot 1/\sqrt{T} = \sqrt{T}$. For a rigorous lower bound, see Lattimore & Szepesvári, *Bandit Algorithms*, 2020.

5.3 Context-dependent Decisions

In the first chapter, we considered decisions which depend on a context, such as in personalizing recommendation to individual users. We now return to this setting, and focus on the online aspect of the problem: when each decision is made using the data collected so far, how quickly do we learn to take good actions? For example, online advertisements can be personalized to user profile and browsing history. Unlike the fixed context setting, there is no single best action, since the best action may be different in every context.

The agent interacts with the environment as follows.

This setting is canonically known as the *contextual bandit* setting.

For $t = 0, \ldots, T-1$:
- Observe context $x_t \sim \mu$
- Choose action $a_t \in [K]$
- Receive reward $r_t \sim R(x_t, a_t)$

Above, the contexts $x_t \in \mathcal{X}$ are drawn independently from a distribution $\mu \in \Delta(\mathcal{X})$, and the reward distribution $R(x, a)$ depends on both the context and the action. We write $r(x, a) = \mathbb{E}_{r \sim R(x,a)}[r]$ for the mean reward. This is the same as the setting of the first chapter, except that we index the data by the time t instead of by i, due to our focus on learning quickly.

The best action in each context is the one with the highest mean reward, so the optimal policy is

$$\pi^\star(x) \in \arg\max_{a \in [K]} r(x, a)\,.$$

The regret compares the agent's actions to this policy,

$$\text{Regret}_T = \sum_{t=0}^{T-1} \big(r(x_t, \pi^\star(x_t)) - r(x_t, a_t)\big)\,.$$

Notice that the benchmark is now a policy, which may choose a different action in every context. If there are only finitely many contexts, one approach is to run a separate copy of UCB in each context. However, this approach learns nothing about one context from data collected in another, so its regret grows with the number of contexts. In most applications the number of possible contexts is very large, but similar contexts call for similar actions.

Let T_x be the number of times context x appears. Then the regret of running a separate copy of UCB in each context grows as $\sum_x \sqrt{KT_x} \le \sqrt{|\mathcal{X}|KT}$, ignoring logarithmic factors, by the Cauchy-Schwarz inequality.

To extend UCB to this setting, we need confidence intervals for the mean reward $r(x, a)$ as a function of the context. For general functions, this is difficult. However, for linear functions, confidence intervals are straightforward to construct. Suppose that contexts are vectors $x \in \mathbb{R}^d$, and that the mean reward is linear in the context,

$$r(x, a) = \theta_a^\top x\,.$$

Above, $\theta_a \in \mathbb{R}^d$ is an unknown parameter vector for each action $a \in [K]$. For example, in music recommendation, the actions may be artists and the contexts may be users. Then the entries of θ_a describe the attributes of artist a (for example, tempo and lyricism), and the entries of x describe how much a user values each of these attributes.

To predict rewards, we estimate each θ_a from the data observed so far, $\{(x_k, a_k, r_k)\}_{k=0}^{t-1}$, using only the data points where action a was chosen. We use regularized least squares,

$$\hat{\theta}_a = \operatorname*{arg\,min}_{\theta \in \mathbb{R}^d} \sum_{k:a_k=a} \big(\theta^\top x_k - r_k\big)^2 + \lambda \|\theta\|_2^2\,.$$

The regularizer $\lambda > 0$ ensures that A_a is invertible, which is important at the start of the interaction, before the contexts in which a was chosen span $\mathbb{R}^d$.

Recall the closed form solution to regularized least squares from our discussion of fitted value iteration. Here, it is

$$\hat{\theta}_a = A_a^{-1} b_a\,, \qquad A_a = \lambda I + \sum_{k:a_k=a} x_k x_k^\top\,, \qquad b_a = \sum_{k:a_k=a} x_k r_k\,.$$

The matrix A_a is closely related to the covariance of the contexts. The following claim gives the form of the confidence interval.

If N_a is the number of times action a has been chosen, then $A_a = \lambda I + N_a \hat{\Sigma}_a$, where $\hat{\Sigma}_a = \frac{1}{N_a} \sum_{k:a_k=a} x_k x_k^\top$ is the empirical covariance of the contexts in which action a was chosen.

Claim: Fix a failure probability $\delta \in (0, 1)$ and a context x. Then there is a constant α, depending on δ, λ, and the reward distribution, such that with probability at least $1 - \delta$,

$$\left|\theta_a^\top x - \hat{\theta}_a^\top x\right| \le \alpha \sqrt{x^\top A_a^{-1} x}\,.$$

In the online setting, the contexts and actions in the data depend on previous rewards, and we need the bound to hold for all contexts and time steps at once, so a more careful argument is needed. The resulting bound has the same form, with α that also grows logarithmically with T. See Chapters 19 and 20 of Lattimore & Szepesvári, *Bandit Algorithms*, 2020.

Exercise 5.5. *Let's justify this claim in a simplified setting. Treat the contexts and actions as fixed, take $\lambda = 0$, and assume that the reward is Gaussian $R(x, a) = \mathcal{N}(\theta_a^\top x, \sigma^2)$. Writing $r_k = \theta_a^\top x_k + w_k$ for $w_k \sim \mathcal{N}(0, \sigma^2)$*

and substituting into the least squares solution, show that

$$\hat{\theta}_a = A_a^{-1} \sum_{k:a_k=a} x_k(x_k^\top \theta_a + w_k) = \theta_a + A_a^{-1} \sum_{k:a_k=a} w_k x_k \,.$$

Then, show that $\hat{\theta}_a^\top x - \theta_a^\top x$ is Gaussian with mean zero and variance $\sigma^2 x^\top A_a^{-1} x$. Use the fact that a Gaussian random variable $z \sim \mathcal{N}(0, s^2)$ satisfies $|z| \leq s\sqrt{2\ln(2/\delta)}$ with probability at least $1-\delta$ to prove the claim. What is α?

As in the fixed context setting, we will act optimistically, this time with respect to the prediction interval defined in the claim. The resulting algorithm is called *LinUCB*. For each action, it keeps track of the matrix A_a and the vector b_a, which can be updated incrementally as data arrives.

Input: regularizer $\lambda > 0$, confidence parameter $\alpha > 0$

Initialize $A_a = \lambda I$, $b_a = 0$ for each $a \in [K]$

For $t = 0, \ldots, T-1$:

Observe context x_t

$$a_t = \operatorname*{arg\,max}_{a \in [K]} \hat{\theta}_a^\top x_t + \alpha\sqrt{x_t^\top A_a^{-1} x_t}\,, \quad \text{where } \hat{\theta}_a = A_a^{-1} b_a$$

Receive reward r_t

$$A_{a_t} \leftarrow A_{a_t} + x_t x_t^\top\,, \quad b_{a_t} \leftarrow b_{a_t} + r_t x_t$$

The upper confidence bound has a geometric interpretation. The first term $\hat{\theta}_a^\top x_t$ encourages exploitation, since it is large if the context is aligned with the estimated parameter. The second term $\alpha\sqrt{x_t^\top A_a^{-1} x_t}$ encourages exploration, since it is large if the context points in a direction which is poorly covered by the data, as illustrated by Figure 5.4. Like UCB, LinUCB explores adaptively, now in a context-dependent manner.

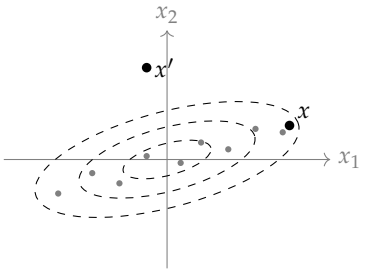


Figure 5.4: Contexts in which action a was chosen (gray), and three level sets of the exploration bonus $(x^\top A_a^{-1} x)^{1/2}$. Since many past contexts lie along the long axis, the bonus grows slowly in that direction: the context x receives less bonus than x'.

Exercise 5.6. *Consider LinUCB with $d = 1$ and contexts which are always $x_t = 1$. Show that, up to the choice of α and the regularization λ, LinUCB is the same as the UCB algorithm from the previous section.*

Where does the prediction interval come from? In the fixed context setting, the upper confidence bound came from a confidence interval on the unknown mean of each action. Here, the prediction interval comes from a confidence set on the unknown parameter θ_a. Since θ_a is d-dimensional, this set is no longer an interval, but an ellipsoid of plausible parameters,

$$\mathcal{C}_a = \left\{\theta : (\theta - \hat{\theta}_a)^\top A_a (\theta - \hat{\theta}_a) \leq \alpha^2\right\},$$

centered at the estimate $\hat{\theta}_a$ (Figure 5.5).

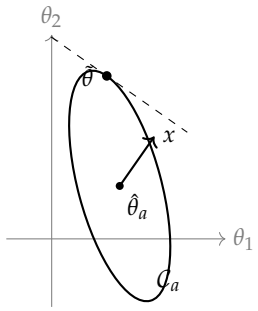


Figure 5.5: The set of plausible parameters $\mathcal{C}_a$, an ellipsoid centered at the estimate $\hat{\theta}_a$, for the same matrix A_a as in Figure 5.4. It is narrow in the direction which is well covered by past contexts, and long in the direction which is not. The dashed line is a level set of the predicted reward $\theta^\top x$ for a context x. The most optimistic plausible parameter $\tilde{\theta}$ is where this line touches the ellipsoid, and its predicted reward $\tilde{\theta}^\top x$ is the upper confidence bound.

Exercise 5.7. *Show that the upper confidence bound is the largest reward predicted by any plausible parameter,*

$$\max_{\theta \in \mathcal{C}_a} \theta^\top x = \hat{\theta}_a^\top x + \alpha \sqrt{x^\top A_a^{-1} x}\,.$$

Under the linear reward model, the regret of LinUCB grows as $\sqrt{T}$, up to logarithmic factors, and linearly in the number of unknown parameters. We will not prove this, though the argument follows the same steps as the analysis of UCB: the regret at each time step is bounded by the width of a confidence interval, and then a clever argument is required to bound the sum of these widths.

The linear reward model is more general than it first appears. It is possible to apply the same ideas to the slightly more general case that the mean reward is linear in some known feature map $\varphi : \mathcal{X} \times \mathcal{A} \to \mathbb{R}^p$,

$$r(x, a) = \theta_\star^\top \varphi(x, a)\,,$$

with a single unknown parameter $\theta_\star \in \mathbb{R}^p$. Then the upper confidence bound for $\varphi = \varphi(x, a)$ is $\hat{\theta}^\top \varphi + \alpha \sqrt{\varphi^\top A^{-1} \varphi}$, where now a single $A = \lambda I + \sum_k \varphi_k \varphi_k^\top$ is shared by all of the actions. The benefit of this model is that data from one action informs the predictions for the others, so the regret depends on the dimension p of the features instead of the number of actions. This makes it possible to handle very large, or even continuous, sets of actions. The downside is that choosing an action requires maximizing the upper confidence bound over all of the actions, which can be computationally expensive when the set of actions is large.

Exercise 5.8. *Show that the fixed context setting is a special case of this model, with $\varphi(a) = e_a \in \mathbb{R}^K$ the ath standard basis vector and that the linear model with a separate parameter for each action, $r(x, a) = \theta_a^\top x$, is a special case with $\varphi(x, a) \in \mathbb{R}^{dK}$. What is $\varphi(x, a)$, and what is $\theta_\star$?*

The algorithms in this chapter follow a common pattern. We model the effect of each action on the reward, possibly depending on the context, along with our uncertainty about this model. We then use the model and its uncertainty together to choose promising actions, exploring where the uncertainty is large and exploiting where the model is confident. In this chapter, the model was a model of the reward. More generally, it is also necessary to learn models of transitions and values.

6
Receding Horizons

So far, we have seen two ways to find good policies. When the reward and transition models are known, dvalue iteration and policy iteration compute near optimal policies. However, these algorithms enumerate every state and action, so they only work in modest tabular settings. When the state space is large or continuous, we turned instead to learning from data, with fitted value iteration, fitted policy iteration, and policy gradients. Is learning from data the only option once the state space is large? In many problems, we have a good model of how states evolve, e.g. known physical dynamics or the rules of a game, or we might learn it from data. In this chapter, we develop methods which use models to act well in large and continuous problems. The key idea is to solve a *proxy* for our real objective, starting from the current state, take an action based on the solution, and then repeat. A key simplification for our proxy objective is to plan over a short horizon. To solve the proxy problems, we will encounter many ideas from earlier chapters, including dynamic programming, value functions, and random search.

6.1 Extending Finite Horizons

We begin by unpacking the effect of planning over short horizons in the tabular setting. Recall problems with a fixed horizon H. Now, we will also allow a discount factor $\gamma \in (0, 1]$ and a *terminal value* $\bar{V}$, which assigns a value to the state reached at the end of the horizon. Recall the dynamic programming algorithm. Starting from $V_H = \bar{V}$, for $t = H-1, \ldots, 0$ we compute

In the fixed horizon setting of the earlier chapters, we used $\gamma = 1$ and $\bar{V} = 0$.

$$Q_t(s,a) = r(s,a) + \gamma \, \mathbb{E}_{s' \sim P(s,a)} \left[V_{t+1}(s') \right],$$
$$\pi_t(s) \in \arg\max_a Q_t(s,a), \qquad V_t(s) = \max_a Q_t(s,a).$$

This gives a time-dependent policy $\pi_0, \ldots, \pi_{H-1}$ which would be optimal for an episode of exactly H steps.

What if the interaction lasts much longer than H steps, or indefinitely? Our simple approach is to keep applying the *first* policy π_0 at every time step. At each time step, we act as if there are H steps remaining, take the first action of the fixed horizon plan, and then repeat. Since the planning horizon always extends H steps beyond the current time, this is called *receding horizon control* (Figure 6.1). The resulting policy $\pi^{\text{RHC}}(s) = \pi_0(s)$ does not depend on time.

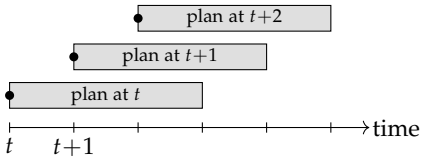


Figure 6.1: Receding horizon control, also known as *model predictive control* (MPC). At each time step, the agent plans $H = 3$ steps ahead from the current state, takes only the first action of the plan (dot), and then plans again.

Receding horizon control is closely related to value iteration. With terminal value $\bar{V}$, dynamic programming is equivalent to performing H steps of value iteration starting from $\bar{V}$. The RHC policy π_0 acts greedily with respect to the resulting $V_1 = (\mathcal{B}^\star)^{H-1}(\bar{V})$,

$$\pi^{\text{RHC}}(s) \in \arg\max_a \; r(s,a) + \gamma \, \mathbb{E}_{s' \sim P(s,a)} \left[V_1(s') \right] .$$

So receding horizon control is the same as running value iteration for $H - 1$ steps, starting from the terminal value, and then acting greedily. In the tabular setting, this does not save any computation compared with value iteration: each of the H steps of dynamic programming costs $O(|\mathcal{S}|^2|\mathcal{A}|)$, just like a step of value iteration. The difference is in how we think about the horizon. Instead of iterating until the values converge, we choose the number of steps H in advance. For now, we focus on how the choice of H and $\bar{V}$ affects the quality of this policy. How large should the horizon be? If it is too short, the planner cannot see the consequences of its actions.

Exercise 6.1. *Consider the chain MDP with states $\mathcal{S} = \{0, 1, \ldots, L\}$ and actions $\mathcal{A} = \{\text{left}, \text{right}\}$. The action* right *moves from s to $\min\{s+1, L\}$, the action* left *moves from s to $\max\{s-1, 0\}$, and the reward is $r(s,a) = 1$ if $s = L$ and 0 otherwise. Let $\gamma = 1$, and suppose that receding horizon control breaks ties uniformly at random.*

1. *With $\bar{V} = 0$, show that receding horizon control chooses* right *at state s if $H > L - s$, and chooses uniformly at random if $H \leq L - s$.*
2. *Now suppose that $\bar{V}(s) = s$. Show that receding horizon control chooses* right *at every state, even with $H = 1$.*

If we did not already know the reward, we could define $\bar{V}$ to add an exploration bonus of 1 for every state s which has not yet been visited. What would happen using receding horizon control with $H = 1$ and starting from state 0?

Good terminal values can make up for short planning horizons.

Exercise 6.2. *Using the bound from the chapter on value iteration for greedy policies, show that for $\gamma < 1$,*

$$\|V^{\pi^{\text{RHC}}} - V^\star\|_\infty \leq \frac{2\gamma^H}{1-\gamma} \|\bar{V} - V^\star\|_\infty .$$

What happens if $\bar{V} = V^\star$? What happens if $\bar{V} = 0$?

With a perfect terminal value $\bar{V} = V^\star$, a horizon of one is enough to be optimal. With no terminal value, the horizon must be long

enough to see the consequences of each action. In practice, the terminal value may be learned from data, for example with fitted value iteration, or a heuristic designed by hand, for example to encourage exploration.

Adding an exploration bonus is the principle of optimism in the face of uncertainty from the previous chapter, now applied to planning. Algorithms based on this idea, such as UCB-VI, have regret guarantees for tabular MDPs (see Azar et al., *Minimax regret bounds for reinforcement learning*, 2017).

Now that we understand the role of the horizon, we can return to the question of computation. Dynamic programming computes π_0 for every state, which is too expensive when the state space is very large. However, at each time step, we only need the first action from the current state. To take advantage of this, we now restrict our attention to deterministic transitions, $s_{t+1} = f(s_t, a_t)$. Then planning from the current state s is the same as optimizing over *open loop* sequences of actions,

$$\max_{a_0,\dots,a_{H-1}} \sum_{k=0}^{H-1} \gamma^k r(s_k, a_k) + \gamma^H \tilde{V}(s_H) \qquad \text{s.t. } s_0 = s_t,\ s_{k+1} = f(s_k, a_k)\,.$$

When the true transitions are stochastic, the deterministic model f may be an approximation. For example, $f(s,a)$ could be the most likely next state $\arg\max_{s'} P(s'|s,a)$, the mean next state, or a sample $s' \sim P(s,a)$ drawn once and then held fixed.

An open loop plan commits to the entire sequence of actions in advance, as if we were acting with our eyes closed. This is similar to the auto-regressive policies from the second chapter, which generate an entire sequence of actions given a context. For deterministic transitions, nothing is lost by "closing our eyes": we can predict every state in advance, so there is nothing new to see. When we incorporate this into receding horizon control, we "open our eyes" at the next time step, and plan again from the observed state.

With stochastic transitions, an open loop sequence of actions is generally suboptimal.

For discrete actions, the simplest way to solve this problem is to try every sequence of actions. The sequences form a tree, rooted at the current state, in which each node branches into $|\mathcal{A}|$ children (Figure 6.2). There are $|\mathcal{A}|^H$ sequences, each of which requires simulating H steps of the dynamics. So the cost of this *brute force tree search* does not depend on the number of states at all, but it grows exponentially with the horizon. In contrast, the $O(H|\mathcal{S}|^2|\mathcal{A}|)$ cost of dynamic programming grows only linearly with the horizon but quadratically with the number of states. Tree search can be preferable when the state space is very large and the horizon is short.

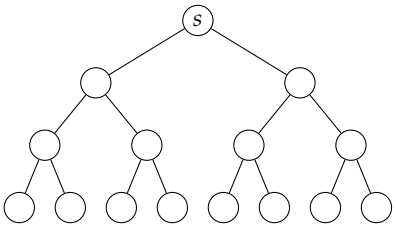


Figure 6.2: The search tree for $|\mathcal{A}| = 2$ and $H = 3$. Each path from the root s to a leaf is a sequence of actions, and there are $|\mathcal{A}|^H = 8$ of them.

Brute force search is clearly naive, since it spends just as much effort on a sequence which begins with an obviously bad action as on a sequence which begins with a promising one. There are many ideas for avoiding searching the entire tree, especially in the application of board games, where the state space is large. *Alpha-beta pruning* skips branches which cannot affect the decision. It was central to chess programs, like Deep Blue, which defeated the world champion Garry Kasparov in 1997. *Monte Carlo tree search* estimates the value of a node by simulating random continuations, and expands the most promising nodes first, using ideas similar to the UCB. Based on ideas applied to chess in the 1980s, Monte Carlo tree search was

developed in the mid 2000s, and it led to Go playing programs at the level of strong amateur players. To reach the level of the best human players, AlphaGo combined Monte Carlo tree search with two learned components: a value function, which plays the role of the terminal value, and a policy, which suggests promising moves to search. In 2016, AlphaGo defeated the Go champion Lee Sedol. Later, MuZero went one step further and learned the model used by the tree search from data, instead of being given the rules of the game, allowing it to play Go, chess, shogi, and Atari games.

6.2 *Continuous Actions*

So far, our algorithms are designed for finite state and action spaces. For continuous states and actions where $\mathcal{S} = \mathbb{R}^{d_s}$ and $\mathcal{A} = \mathbb{R}^{d_a}$, dynamic programming requires representing a value function over infinitely many states, and tree search requires branching over infinitely many actions. Instead, we develop a different approach from the open-loop view. Just as before, receding horizon control solves the open loop problem

One option is to discretize: round each coordinate of the state or action to a grid of width ϵ. If the states lie in $[-1, 1]^{d_s}$, this gives $(2/\epsilon)^{d_s}$ states, which grows exponentially with the dimension. For example, $\epsilon = 0.01$ and $d_s = 10$ gives $200^{10} \approx 10^{23}$ states. Discretizing the states is too expensive for tabular methods like dynamic programming, and discretizing the actions in the same way is too expensive for tree search.

$$\max_{a_0,\ldots,a_{H-1}} \sum_{k=0}^{H-1} r(s_k, a_k) + \bar{V}(s_H) \qquad \text{s.t. } s_0 = s_t,\ s_{k+1} = f(s_k, a_k)$$

at each time step, takes the first action, and repeats.

For simplicity, we drop the discount factor in the rest of this chapter, which is the convention in control. All of the ideas apply equally with discounting.

Now, this is a continuous optimization problem over the Hd_a variables $\mathbf{a} = (a_0, \ldots, a_{H-1}) \in \mathbb{R}^{Hd_a}$. Substituting the dynamics, each state s_k is a function of the initial state and the previous actions, so the objective is a function $J(s_t, \mathbf{a})$ of the initial state and actions alone. If the dynamics and rewards are differentiable, we can maximize it with gradient ascent, computing the gradient with automatic differentiation through the simulated trajectory. In receding horizon control, since the optimization is repeated at every time step, it is common to *warm start* or initialize any iterative optimization algorithm from the previous plan, shifted forward by one step.

This is the same computation as backpropagation through time for recurrent neural networks. When the number of variables Hd_a is small enough, second order methods, such as Newton's method or quasi-Newton methods like L-BFGS, can converge much faster.

An advantage of the continuous optimization view is that it can include constraints, such as limits on the actuators or safety constraints on the states,

$$s_k \in \mathcal{S}_{\text{safe}}\,, \qquad a_k \in \mathcal{A}_{\text{feas}}\,.$$

Plain gradient ascent does not handle constraints well, so constrained problems are usually solved with specialized optimization methods. When the dynamics are linear, the rewards are concave quadratic functions, and the constraints are linear, the open loop problem is a *quadratic program*, which can be solved efficiently and reliably with off-the-shelf solvers. Model predictive control of this form has been

used in the process control industry, for example in oil refineries and chemical plants, since the 1980s.

Exercise 6.3. *Consider a robot moving along a line with $s_{k+1} = s_k + a_k$ and reward $r(s, a) = -s^2 - a^2$, with the actuator constraint $|a_k| \leq 1$. Write the open loop problem as a quadratic program in the variables $a_0, \ldots, a_{H-1}$.*

Even when the dynamics are not linear or the rewards are not quadratic, we can approximate them with a first order approximation of the dynamics, using Jacobians, and a second order approximation of the rewards, using Hessians. The resulting quadratic program can be solved, and then the approximation is repeated around the new solution. We will explore this idea in more detail below

Repeatedly solving a quadratic approximation of a constrained problem is known as *sequential quadratic programming* (SQP).

When the dynamics or rewards are not differentiable, we can instead optimize over action sequences by sampling, treating the objective $J(s, \mathbf{a})$ as a black box which we can only evaluate. This is similar to the policy optimization methods from the first chapter, which also only required evaluating the reward of sampled actions. The *cross-entropy method* (CEM) samples action sequences from a Gaussian distribution, evaluates each one by simulating the model, and refits the Gaussian to the best performing sequences. After several iterations, the mean of the Gaussian is used as the plan, and receding horizon control takes its first action. *Model predictive path integral control* (MPPI) instead updates the mean using a weighted average of all samples, with weights proportional to $\exp(J(s, \mathbf{a}_i)/\lambda)$. Again, the action taken is the first entry of the updated mean. Both methods are policy optimization methods for the class of open loop policies. Like the random search methods from the first chapter, CEM and MPPI perturb the parameters, observe the resulting values, and use only evaluations of the model to decide how to update the parameters. The difference from the first chapter is that the "environment" is a simulation with models, so evaluating a policy does not require interacting with the real world.

6.3 Linear Quadratic Problems

When does the continuous open loop problem have a closed form solution? Consider states $s \in \mathbb{R}^{d_s}$, actions $a \in \mathbb{R}^{d_a}$, linear dynamics, and negative quadratic rewards,

Everything in this section extends to time varying dynamics and rewards, with A_t, B_t, Q_t, and R_t in place of A, B, Q, and R.

$$s_{t+1} = As_t + Ba_t\,, \quad r(s, a) = -s^\top Qs - a^\top Ra\,, \quad \bar{V}(s) = -s^\top Q_H s\,,$$

where $Q, Q_H \succeq 0$ and $R \succ 0$, and there are no constraints. Maximizing the total reward is the same as minimizing the total cost $\sum_k (s_k^\top Q s_k + a_k^\top R a_k) + s_H^\top Q_H s_H$, and following the convention in con-

trol, we will work with this minimization problem. This is the *linear quadratic regulator* (LQR).

Example 6.1. Consider a drone which can apply an upward or downward thrust. Its state $s = (p, v)$ consists of its height p relative to a target height and its vertical velocity v. The thrust a results in acceleration, changing the velocity. Over one time step, the update is

$$s_{t+1} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} s_t + \begin{bmatrix} 0 \\ 1 \end{bmatrix} a_t \,.$$

To keep the drone near the target without using too much thrust, we may choose the cost $p^2 + a^2$, which corresponds to $Q = \mathrm{diag}(1, 0)$ and $R = 1$.

Since the dynamics are linear, each state is a linear function of the initial state and the actions,

$$s_k = A^k s_0 + \sum_{j=0}^{k-1} A^{k-1-j} B a_j \,.$$

Substituting into the costs, the objective is a quadratic function of the stacked actions $\mathbf{a} = (a_0, \dots, a_{H-1})$,

$$J(s_0, \mathbf{a}) = \mathbf{a}^\top M \mathbf{a} + 2 s_0^\top N \mathbf{a} + s_0^\top L s_0$$

for some matrices $M \succ 0$, N, and L which depend on A, B, Q, R, Q_H. Setting the gradient to zero gives $\mathbf{a}^\star = -M^{-1} N^\top s_0$, which is linear in the initial state. So the first action is also linear in the current state, and receding horizon control is a linear policy $a_t = -K^{\mathrm{RHC}} s_t$. In the drone example, this means that the thrust is a weighted combination of the height and the velocity.

Stacking the states, $\mathbf{s} = (s_0, \dots, s_H) = G s_0 + F \mathbf{a}$, where G stacks the matrices $I, A, \dots, A^H$, and F is block lower triangular with block (k, j) equal to $A^{k-1-j} B$ for $j < k$. With $\bar{Q} = \mathrm{diag}(Q, \dots, Q, Q_H)$ and $\bar{R} = \mathrm{diag}(R, \dots, R)$, the matrices are $M = \bar{R} + F^\top \bar{Q} F$, $N = G^\top \bar{Q} F$, and $L = G^\top \bar{Q} G$.

This approach requires solving a linear system of dimension $H d_a$, which naively costs $O(H^3 d_a^3)$. It also does not tell us much about the structure of the solution, beyond the fact that it is linear. Dynamic programming takes advantage of the structure of the problem to reduce the cost to one which is linear in H, and it also makes clear the role of the terminal cost.

Claim: The optimal cost-to-go is quadratic and the optimal policy is linear,

$$V_t(s) = s^\top P_t s \,, \qquad \pi_t(s) = -K_t s \,,$$

where $P_H = Q_H$ and, for $t = H - 1, \dots, 0$,

$$K_t = (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A \,,$$
$$P_t = Q + A^\top P_{t+1} A - A^\top P_{t+1} B (R + B^\top P_{t+1} B)^{-1} B^\top P_{t+1} A \,.$$

The same policy is optimal when the dynamics include additive noise, $s_{t+1} = A s_t + B a_t + w_t$, with w_t independent, mean zero, and covariance Σ. The noise only adds the constant $\mathrm{tr}(\Sigma P_{t+1})$ to the cost-to-go at each step, which does not affect the minimizing action.

The second equation is known as the *Riccati equation*.

Exercise 6.4. *Prove the claim by induction, From $V_{t+1}(s) = s^\top P_{t+1} s$,*

$$V_t(s) = \min_a \; s^\top Q s + a^\top R a + (As + Ba)^\top P_{t+1} (As + Ba) \,,$$

then minimize over a by setting the gradient to zero, and substitute the minimizer back in.

For receding horizon control, the claim means that we never need to solve the planning problem from scratch. We run the Riccati equation for H steps, starting from $P_H = Q_H$, to compute the gain K_0. Since the dynamics and costs do not depend on time, the planning problem is the same at every time step, so $K^{\text{RHC}} = K_0$ can be computed once in advance. Then at each time step, the action is a single matrix-vector product, $a_t = -K_0 s_t$.

Exercise 6.5. *Consider a robot moving along a line with $s_{t+1} = s_t + a_t$ and cost $s^2 + a^2$, i.e. $d_s = d_a = 1$ and $A = B = Q = R = 1$, with terminal cost $Q_H = 0$. Show that the gains satisfy $K_{H-1} = 0$ and $K_t = \frac{1+K_{t+1}}{2+K_{t+1}}$, so that $K_{H-2} = \frac{1}{2}$, $K_{H-3} = \frac{3}{5}$, $K_{H-4} = \frac{8}{13}$. What does K_0 converge to as $H \to \infty$?*

As in the tabular setting, the terminal cost plays the role of the initial guess in value iteration, and the Riccati equation plays the role of the Bellman operator. Each step of the Riccati equation moves the matrix P_t one step further from the terminal cost Q_H, so the gain K_0 used by receding horizon control depends on both the horizon and the terminal cost. As the horizon grows, the influence of the terminal cost fades, and the gain approaches that of the infinite horizon problem. The proof is out of scope, but for the infinite horizon problem, it can be shown that the optimal cost-to-go is $V^\star(s) = s^\top P^\star s$, where $P^\star$ is a fixed point of the Riccati equation, and that the optimal policy is linear with gain $K^\star = (R + B^\top P^\star B)^{-1} B^\top P^\star A$. Notice that if we choose the terminal cost $Q_H = P^\star$, then every step of the Riccati equation returns $P^\star$ again, since it is a fixed point, resulting in the gain $K^\star$ for any horizon $H \geq 1$. With the right terminal cost, receding horizon control is optimal, even with a horizon of one.

The fixed point equation,

$$P = Q + A^\top P A - A^\top P B (R + B^\top P B)^{-1} B^\top P A\,,$$

is known as the *discrete algebraic Riccati equation*. Under mild conditions on (A, B), the Riccati iterates converge to $P^\star$ from any $Q_H \succeq 0$, just as value iteration converges from any initial guess.

Exercise 6.6. *Consider the robot setting from the previous exercise. What happens to the robot under receding horizon control with $H = 1$? Would a nonzero terminal cost fix this?*

For some systems, short horizons without a terminal cost can even lead to unstable closed loop dynamics, in which the state grows without bound.

Most systems of interest have nonlinear dynamics $s_{t+1} = f(s_t, a_t)$ and non-quadratic costs $c(s, a)$. We can still take advantage of the LQR solution by approximating the problem locally. Given an initial trajectory $(\bar{s}_k, \bar{a}_k)_{k=0}^H$, we consider small deviations $\delta s_k = s_k - \bar{s}_k$ and $\delta a_k = a_k - \bar{a}_k$. A first order Taylor approximation of the dynamics around the trajectory gives

The initial trajectory is often the plan from the previous time step, shifted forward by one.

$$\delta s_{k+1} \approx A_k \delta s_k + B_k \delta a_k\,, \qquad A_k = \nabla_s f(\bar{s}_k, \bar{a}_k)\,, \quad B_k = \nabla_a f(\bar{s}_k, \bar{a}_k)\,,$$

where A_k and B_k are the Jacobians of the dynamics with respect to

the state and action. A second order Taylor approximation of the cost

$$c(s_k, a_k) \approx c(\bar{s}_k, \bar{a}_k) + q_k^\top \delta s_k + r_k^\top \delta a_k + \frac{1}{2} \begin{bmatrix} \delta s_k \\ \delta a_k \end{bmatrix}^\top \begin{bmatrix} Q_k & M_k \\ M_k^\top & R_k \end{bmatrix} \begin{bmatrix} \delta s_k \\ \delta a_k \end{bmatrix},$$

where the gradients and the blocks of the Hessian are

$$q_k = \nabla_s c(\bar{s}_k, \bar{a}_k), \qquad Q_k = \nabla^2_{ss} c(\bar{s}_k, \bar{a}_k), \qquad M_k = \nabla^2_{sa} c(\bar{s}_k, \bar{a}_k),$$
$$r_k = \nabla_a c(\bar{s}_k, \bar{a}_k), \qquad R_k = \nabla^2_{aa} c(\bar{s}_k, \bar{a}_k).$$

When f and c can only be queried, these derivatives can be estimated by *finite differences*, e.g.

$$\frac{\partial}{\partial s_i} f(s, a) \approx \frac{f(s + \epsilon e_i, a) - f(s, a)}{\epsilon}.$$

This approximate problem is a time-varying LQR problem in the deviations $(\delta s_k, \delta a_k)$, with two differences: the cost has a cross term $\delta s_k^\top M_k \delta a_k$, and it has linear terms $q_k^\top \delta s_k + r_k^\top \delta a_k$. Nonetheless, the same dynamic programming argument applies, with a time-varying version of the Riccati equation. The cost-to-go is a quadratic function plus a linear function of the state, and the minimizing action is *affine*, $\delta a_k = -K_k \delta s_k - \kappa_k$. The offset κ_k moves the actions in the direction which the linear terms indicate will decrease the cost, while the gain K_k corrects for deviations of the state from the trajectory.

The Hessian may not be positive definite, in which case the minimization over actions is not well defined. Implementations typically add a multiple of the identity to R_k.

We then use this policy to compute a new trajectory: starting from s_0, we take actions $a_k = \bar{a}_k - K_k(s_k - \bar{s}_k) - \alpha \kappa_k$ and simulate the true nonlinear dynamics. The step size $\alpha \in (0, 1]$ controls how far the new trajectory moves from the old one. The step size is chosen by a line search, so that the new trajectory has lower cost than the old one. It cannot be too big because the approximation is only accurate near the old trajectory. Repeating these steps until the trajectory stops changing gives an algorithm known as *iterative LQR* (iLQR). A closely related algorithm, *differential dynamic programming* (DDP), follows the same steps, except that it also keeps the second order terms in the approximation of the dynamics.

In summary, the algorithm is:

- Initialize trajectory $(\bar{s}_k, \bar{a}_k)$
- Repeat until convergence:
  - Approximate around $(\bar{s}_k, \bar{a}_k)$
  - Solve for (K_k, κ_k)
  - Line search over α and update $(\bar{s}_k, \bar{a}_k)$

In summary, receding horizon control plans from the current state at each time step. The better the terminal value, the shorter the horizon can be. For linear dynamics and quadratic costs, exact dynamic programming solutions are efficient. Otherwise, we may approximate the planning problem with tree search, continuous optimization, or sampling. Throughout this chapter, we assumed that the transition and reward models are correct. In practice, models are usually approximate. Since receding horizon control observes and accounts for the true state at every time step, even a coarse model can lead to a reasonable policy. Though perhaps not optimal, such a policy can be a good starting point for learning. And learned components can be incorporated back into receding horizon control: transition and reward models can be learned, a learned value function can serve as the terminal value, a learned policy can suggest which actions to search.